

Task 1 – Enterprise File Share and NTFS Permission Administration

Objective

Configure and validate a centralized departmental file-sharing structure on **FILESRV01** using SMB share permissions, NTFS permissions, Active Directory security groups, and end-user access testing.

The goal was to implement an enterprise-style access model in which domain users can connect to a common network share while NTFS permissions control access to individual departmental folders.

Environment

File Server: FILESRV01

Domain: BENLAB.LOCAL

Share Name: EnterpriseData

Local Path: C:\EnterpriseData

UNC Path: \\FILESRV01\EnterpriseData

Departmental folders:

- Finance
 - Human Resources
 - Information Technology
 - Public
-

Step 1 – Create the Enterprise Data Share

The EnterpriseData SMB share was created on FILESRV01 using PowerShell.

```
New-SmbShare -Name "EnterpriseData" -Path "C:\EnterpriseData" -ChangeAccess "BENLAB\Domain Users" -FullAccess "BENLAB\Domain Admins"
```

The share configuration was verified with:

```
Get-SmbShare -Name "EnterpriseData"
```

Result: EnterpriseData successfully mapped to C:\EnterpriseData.

Step 2 – Configure and Verify Share Permissions

The SMB share was configured with the following permissions:

Security Principal	Share Permission
BENLAB\Domain Admins	Full
BENLAB\Domain Users	Change

Verification command:

```
Get-SmbShareAccess -Name "EnterpriseData"
```

This allows authenticated domain users to connect to the share while preserving full administrative control for Domain Admins.

Step 3 – Configure Departmental NTFS Permissions

NTFS permissions were used to provide more granular access within the EnterpriseData share.

The resulting departmental access model was:

Folder	Authorized Group	NTFS Permission
Finance	BENLAB\GG_Finance	Modify
Human Resources	BENLAB\GG_HR	Modify
Information Technology	BENLAB\GG_IT	Modify
Public	BENLAB\Domain Users	Modify

Administrative and system permissions were also preserved, including Full Control for SYSTEM and Administrators.

Step 4 – Verify NTFS Permissions with PowerShell

The departmental ACLs were validated using `icacls`.

Example:

```
icacls "C:\EnterpriseData\Finance"
```

A PowerShell loop was also used to inspect all departmental folders:

```
Get-ChildItem "C:\EnterpriseData" -Directory | ForEach-Object {  
    Write-Host "`n=== ${_.Name} ==="  
    icacls $_.FullName  
}
```

The results confirmed that each department was assigned to its corresponding Active Directory security group with **Modify** permission.

Step 5 – Test SMB Connectivity from WIN01

Testing was performed from the domain-joined Windows 11 workstation **WIN01**.

Connectivity to FILESRV01 was verified:

```
ping FILESRV01
```

The server name successfully resolved to:

```
10.20.20.50
```

SMB connectivity was then tested on TCP port 445:

```
Test-NetConnection FILESRV01 -Port 445
```

Result:

```
TcpTestSucceeded : True
```

This confirmed that WIN01 could communicate with the SMB service running on FILESRV01.

Step 6 – Validate Access as a Finance User

Testing was performed while signed in as domain user **jlee**, a Finance employee.

The user successfully browsed:

```
\\FILESRV01\EnterpriseData
```

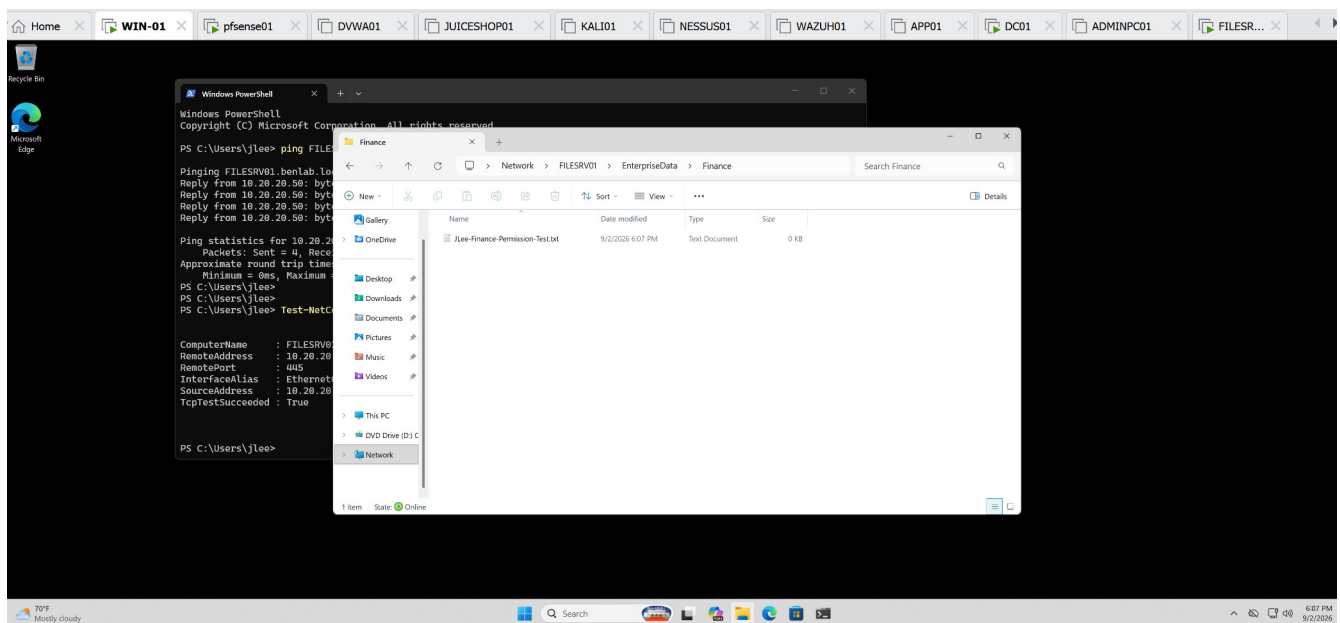
Because jlee is authorized through the Finance security group, the user successfully opened:

```
\\FILESRV01\EnterpriseData\Finance
```

A test file was successfully created:

```
JLee-Finance-Permission-Test.txt
```

This demonstrated that the Finance NTFS **Modify** permission was functioning correctly.



j lee successfully inside Finance showing Jlee-Finance-Permission-Test.txt.

Step 7 – Validate Unauthorized Department Access

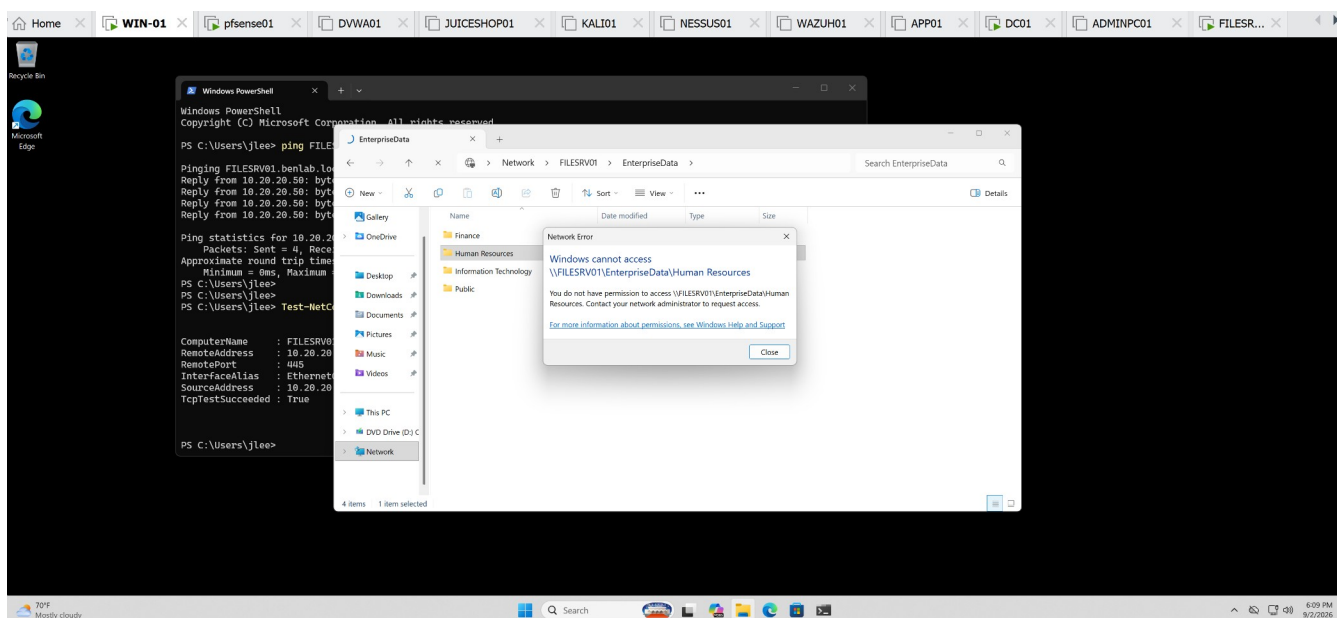
The same Finance user attempted to access:

\\FILESRV01\EnterpriseData\Human Resources

Result: Access denied.

The user also attempted to access:

\\FILESRV01\EnterpriseData\Information Technology



Result: Access denied.

These tests confirmed that having access to the EnterpriseData SMB share does **not** automatically provide access to every folder contained within it.

Departmental access remained controlled by NTFS permissions and Active Directory security-group membership.

Step 8 – Validate Public Folder Access

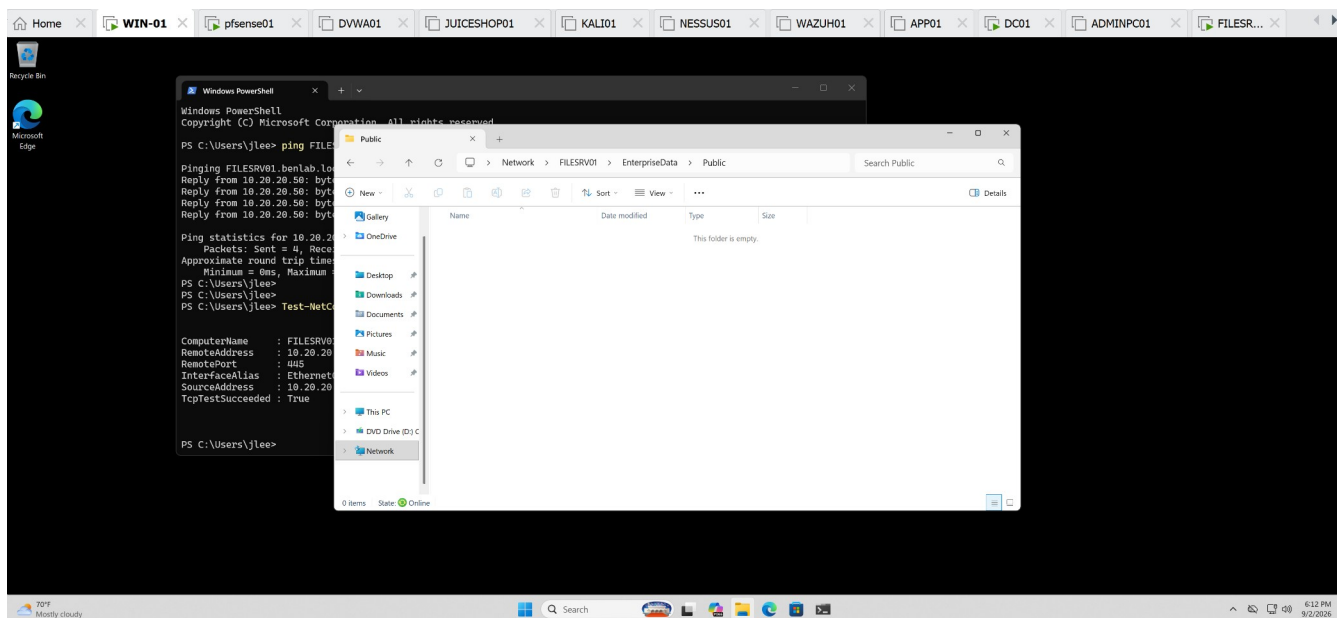
The Finance user successfully accessed:

\\FILESRV01\EnterpriseData\Public

A test file was successfully created:

JLee-Public-Permission-Test.txt

This was expected because the Public folder grants **Modify** permission to BENLAB\Domain Users.



Final Validation

The completed configuration demonstrated the interaction between the two Windows file-access permission layers:

Share permissions

Control access when users connect to the resource across the network.

Domain Users → Change

NTFS permissions

Control what authenticated users can access within the underlying folder structure.

For example:

GG_Finance → Finance → Modify

Therefore, a Finance employee can reach the `EnterpriseData` share but cannot automatically access HR or IT data.

The effective access observed during testing matched the intended security design.

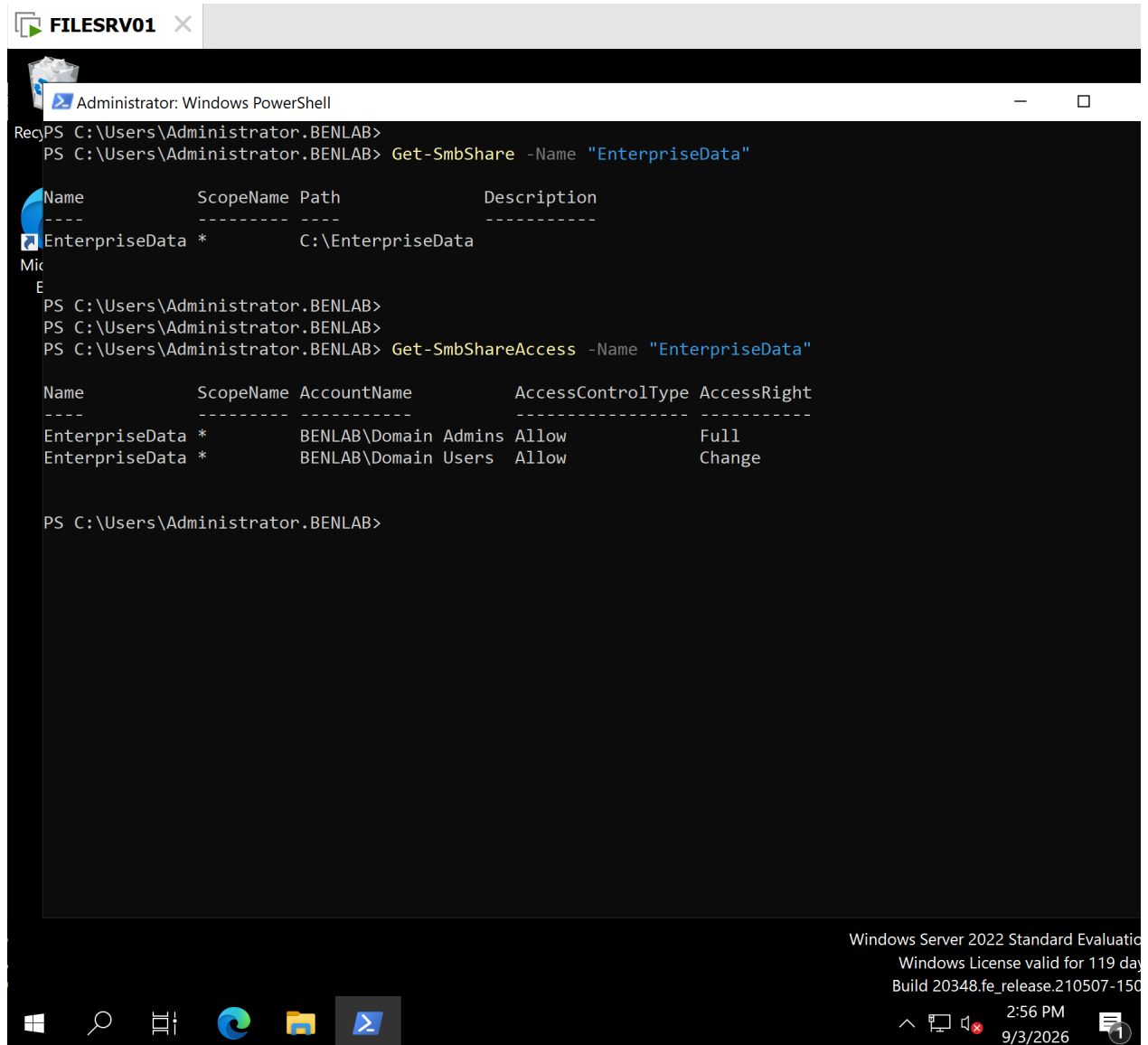
Key Learning Points

- SMB share permissions and NTFS permissions are separate authorization layers.
 - Share permissions provide network-level access to a shared resource.
 - NTFS permissions provide granular file and folder authorization.
 - Active Directory security groups simplify departmental permission management.
 - Users should normally receive access through security-group membership rather than individual user ACL entries.
 - `Get-SmbShare`, `Get-SmbShareAccess`, and `icacls` provide useful administrative validation of file-server permissions.
 - `Test-NetConnection -Port 445` can verify SMB network connectivity before troubleshooting permissions.
 - Successful network connectivity does not necessarily mean a user has authorization to a folder.
 - Testing with a real standard domain user is important because administrative testing alone does not demonstrate the actual end-user permission experience.
-

Task Result

A centralized SMB share was successfully implemented on `FILESRV01` with departmental NTFS authorization based on Active Directory security groups. Access was validated from `WIN01` using a Finance domain user, including successful authorized access, successful file creation, and expected access-denied results for unauthorized departmental resources.

Screenshots / Evidence



Get -SmbShare + Get -SmbShareAccess.

FILESRV01

Administrator: Windows PowerShell

```
Recy=== Public === icac ls C:\EnterpriseData\Public
PS C:\Users\Administrator.BENLAB> Get-ChildItem "C:\EnterpriseData" -Directory | ForEach-Object { Write-Host
    "`n=== $($_.Name) ==="; icac ls $_.FullName }

=== Finance ===
C:\EnterpriseData\Finance BENLAB\GG_Finance:(OI)(CI)(M)
NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
BUILTIN\Administrators:(I)(OI)(CI)(F)
CREATOR OWNER:(I)(OI)(CI)(IO)(F)

Successfully processed 1 files; Failed processing 0 files

=== Human Resources ===
C:\EnterpriseData\Human Resources BENLAB\GG_HR:(OI)(CI)(M)
NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
BUILTIN\Administrators:(I)(OI)(CI)(F)
CREATOR OWNER:(I)(OI)(CI)(IO)(F)

Successfully processed 1 files; Failed processing 0 files

=== Information Technology ===
C:\EnterpriseData\Information Technology BENLAB\GG_IT:(OI)(CI)(M)
NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
BUILTIN\Administrators:(I)(OI)(CI)(F)
CREATOR OWNER:(I)(OI)(CI)(IO)(F)

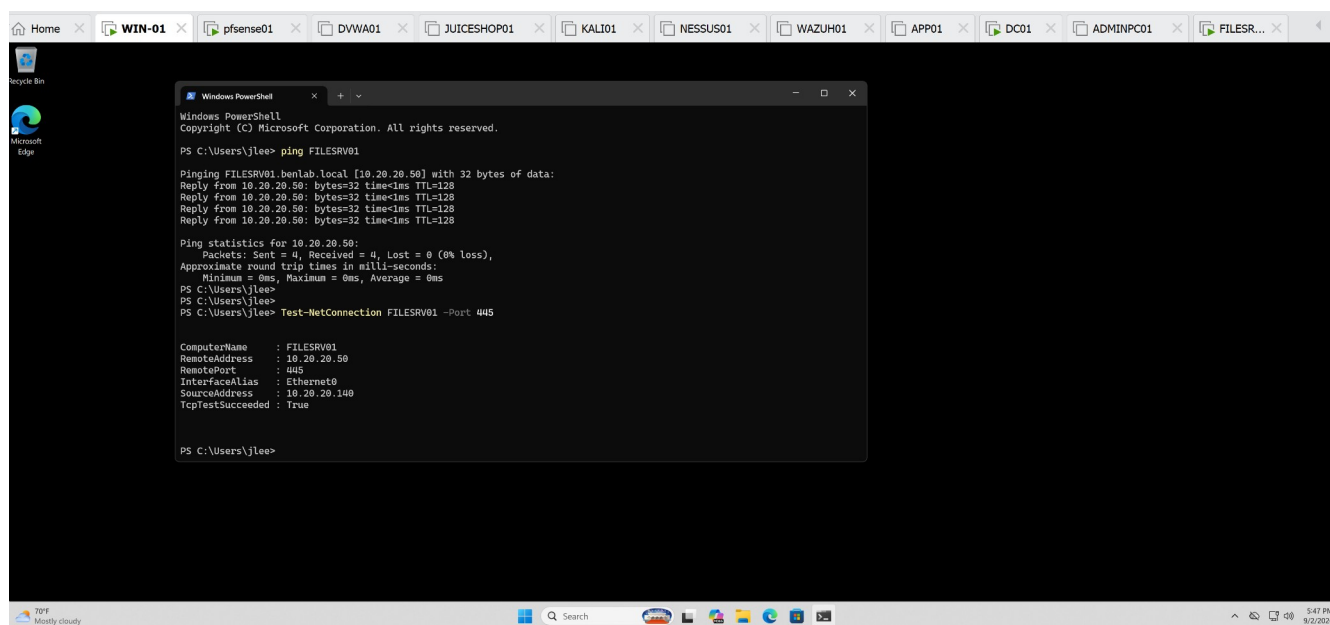
Successfully processed 1 files; Failed processing 0 files

=== Public ===
C:\EnterpriseData\Public BENLAB\Domain Users:(OI)(CI)(M)
NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
BUILTIN\Administrators:(I)(OI)(CI)(F)
CREATOR OWNER:(I)(OI)(CI)(IO)(F)

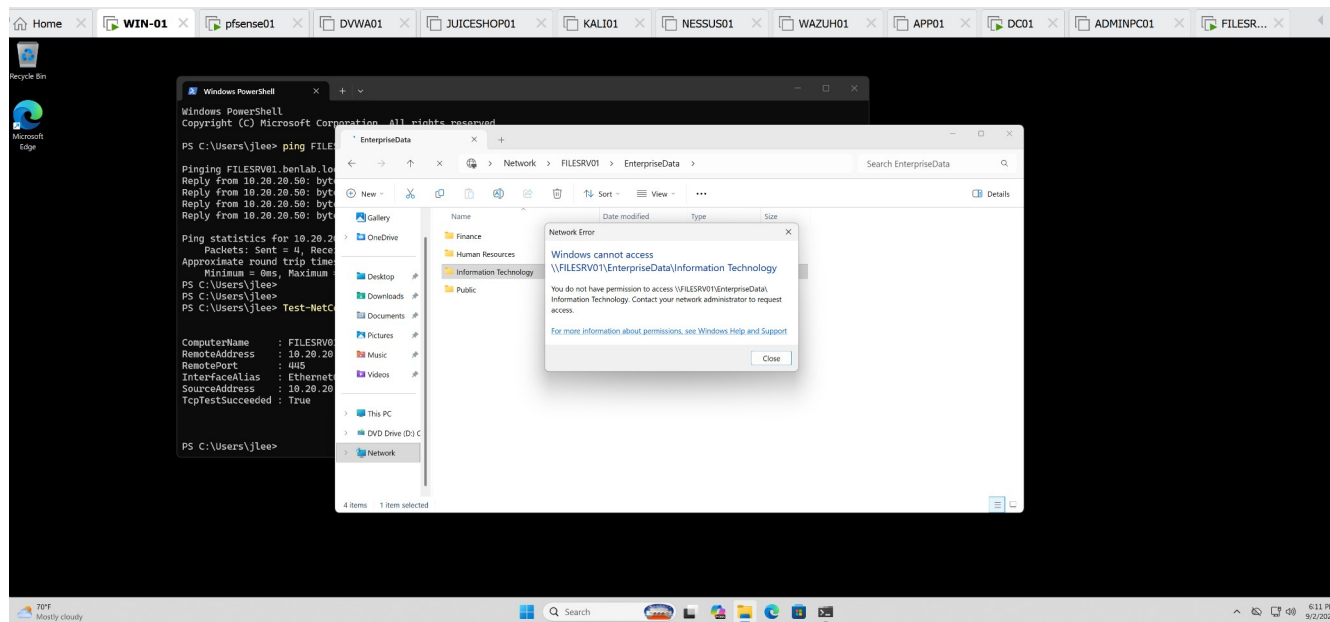
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\Administrator.BENLAB>
```

Windows Server 2022 Standard Evaluation
Windows License valid for 119 days
Build 20348.fe_release.210507-1500
2:46 PM
9/3/2026

The combined `icac ls` output showing **Finance**, **HR**, **IT**, and **Public** permissions.



Test-NetConnection FILESRV01 -Port 445 showing TcpTestSucceeded : True.



Jlee access denied onto Information Technology folder—Windows cannot access.

Task 2 – File Server Resource Manager (FSRM) and Storage Quota Management

Objective

Install and configure **File Server Resource Manager (FSRM)** on FILESRV01 to implement centralized storage quota management.

The task demonstrates how a Windows Server administrator can use quota templates to control departmental storage consumption, generate administrative warnings as storage approaches capacity, and enforce hard storage limits.

A custom **1 GB hard quota template** was created and applied to the Finance departmental folder:
C:\EnterpriseData\Finance

The configuration was then validated through both the FSRM management console and PowerShell, followed by an enforcement test.

Environment

Server: FILESRV01

Domain: BENLAB.LOCAL

Operating System: Windows Server 2022

Role/Feature: File Server Resource Manager (FSRM)

Managed Folder: C:\EnterpriseData\Finance

Quota Limit: 1 GB

Quota Type: Hard Quota

Warning Threshold: 85%

Notification Method: Windows Event Log

Implementation Steps

Step 1 – Verify FSRM Installation Status

PowerShell was used to determine whether File Server Resource Manager was already installed:

```
Get-WindowsFeature FS-Resource-Manager
```

The initial result showed:

[] File Server Resource Manager
Install State: Available

This confirmed that FSRM was available on FILESRV01 but had not yet been installed.

Step 2 – Install File Server Resource Manager

FSRM and its management tools were installed using:

```
Install-WindowsFeature FS-Resource-Manager -IncludeManagementTools
```

The installation completed successfully with:

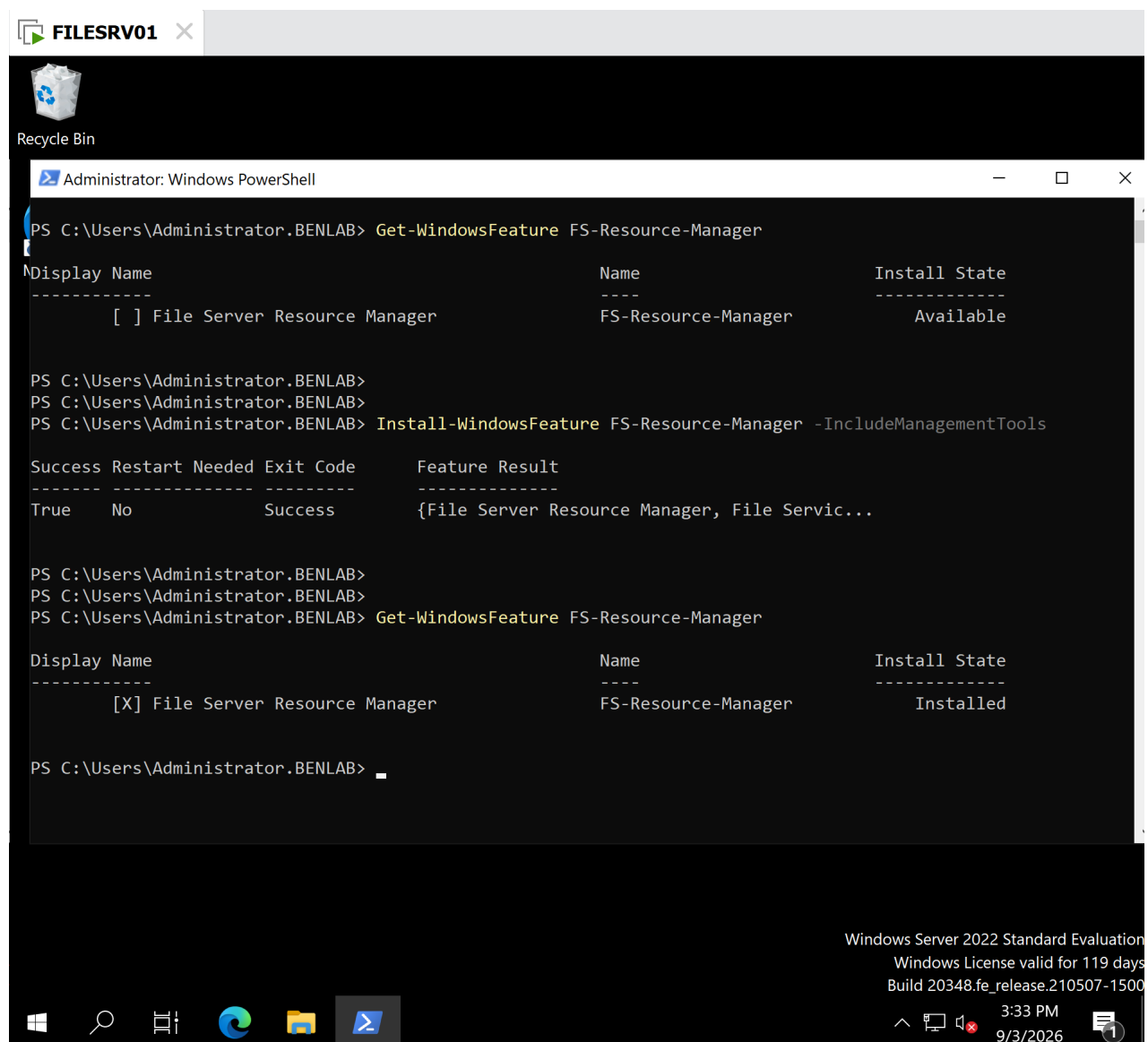
```
Success: True  
Restart Needed: No  
Exit Code: Success
```

The installation was then verified:

```
Get-WindowsFeature FS-Resource-Manager
```

The result showed:

```
[X] File Server Resource Manager  
Install State: Installed
```



This confirmed that FSRM and its management interface were available for configuration.

Step 3 – Open and Review File Server Resource Manager

The **File Server Resource Manager** console was opened on FILESRV01.

The console provides several administrative capabilities, including:

- Quota Management
- File Screening Management
- Storage Reports Management
- Classification Management
- File Management Tasks

For this task, **Quota Management** was used.

Step 4 – Create a Custom Departmental Quota Template

Under:

Quota Management
└─ Quota Templates

a new custom quota template was created.

Template Name:

BENLAB Department 1 GB Hard Quota

Description:

Standard 1 GB storage quota for BENLAB departmental folders

Space Limit:

1 GB

Quota Type:

Hard quota: Do not allow users to exceed limit

A hard quota was intentionally selected because it provides actual enforcement rather than simply monitoring storage consumption.

Step 5 – Configure the Storage Warning Threshold

A notification threshold was configured at:

85%

The **Event Log** notification option was enabled.

This means that when storage consumption reaches 85% of the quota, FSRM can generate a warning in the Windows Event Log, providing administrators with an opportunity to investigate storage consumption before the hard limit is reached.

The completed template therefore provided:

1 GB Hard Limit
└─ 85% → Event Log Warning
└─ 100% → Additional writes exceeding quota blocked

Step 6 – Apply the Quota to the Finance Department

The custom template was applied to:

C:\EnterpriseData\Finance

The following option was selected:

Create quota on path

The quota properties were derived from:

BENLAB Department 1 GB Hard Quota

Before creation, FSRM displayed the configuration summary:

Quota: C:\EnterpriseData\Finance
Source template: BENLAB Department 1 GB Hard Quota
Limit: 1.00 GB (Hard)
Warning (85%): Event log

The quota was then created successfully.

PowerShell Validation

The resulting quota configuration was independently verified from PowerShell:

```
Get-FsrmQuota -Path "C:\EnterpriseData\Finance"
```

Important results included:

```
Disabled          : False  
MatchesTemplate   : True  
Path              : C:\EnterpriseData\Finance  
Size              : 1073741824  
SoftLimit         : False  
Template          : BENLAB Department 1 GB Hard Quota
```

These values confirmed several important configuration details.

MatchesTemplate : True confirmed that the Finance quota remained associated with the custom FSRM template.

Size : 1073741824 represents exactly **1 GiB (1,073,741,824 bytes)**.

SoftLimit : False confirmed that this was an enforced **hard quota**, rather than a monitoring-only soft quota.

Quota Enforcement Test

To verify that the quota actually enforced the configured storage restriction, an oversized test file was attempted inside the Finance folder:

```
fsutil file createnew "C:\EnterpriseData\Finance\Quota-Test.bin" 11000000000
```

The requested test file was approximately 1.1 billion bytes, which exceeded the available space permitted by the 1 GB hard quota.

Windows returned:

Error: There is not enough space on the disk.

The operation was therefore rejected.

This demonstrated that the quota was not merely reporting storage usage—the **hard quota was actively enforcing the configured storage limit**.

Verify No Test File Remained

After the rejected operation, PowerShell was used to verify that the failed test had not left the intended file behind:

```
Test-Path "C:\EnterpriseData\Finance\Quota-Test.bin"
```

Result:

False

This confirmed that Quota-Test.bin did not exist after the failed creation attempt.

Validation / Results

Task 2 successfully demonstrated the complete FSRM quota-management lifecycle:

Install → Configure → Apply → Verify → Test → Validate

The final configuration was:

Configuration	Result
FSRM	Installed
Custom quota template	Created
Template	BENLAB Department 1 GB Hard Quota
Managed folder	C:\EnterpriseData\Finance
Storage limit	1 GB

Configuration	Result
Quota type	Hard
Warning threshold	85%
Warning action	Event Log
Template association	Verified
PowerShell validation	Successful
Oversized write	Blocked
Test-file cleanup check	File absent

Screenshots / Evidence

```

FILESRV01 x
Recycle Bin
Administrator: Windows PowerShell

PS C:\Users\Administrator.BENLAB> Get-WindowsFeature FS-Resource-Manager

Display Name                                     Name                               Install State
-----
[ ] File Server Resource Manager                FS-Resource-Manager                Available

PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB> Install-WindowsFeature FS-Resource-Manager -IncludeManagementTools

Success Restart Needed Exit Code      Feature Result
-----
True      No              Success      {File Server Resource Manager, File Servic...

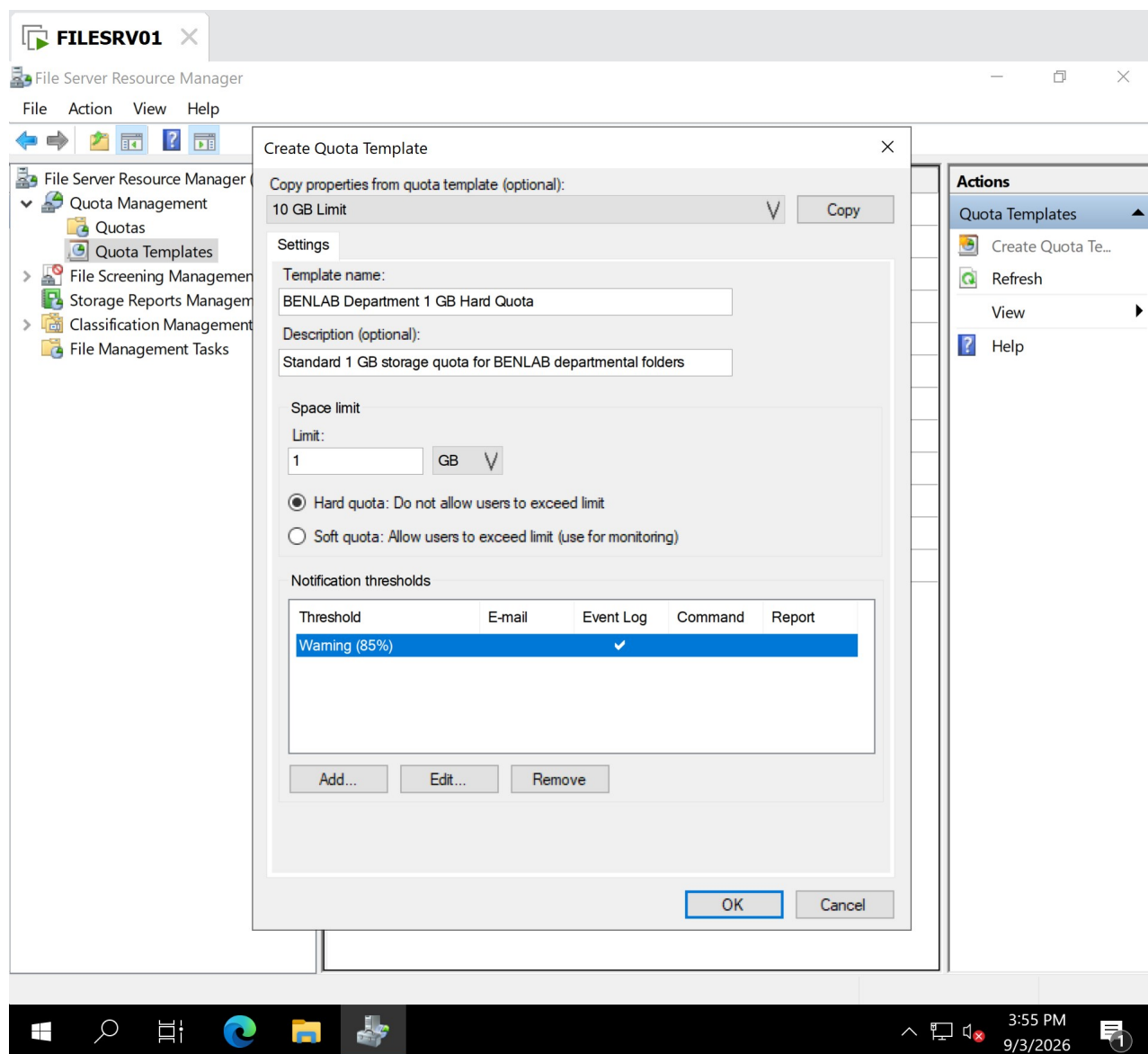
PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB> Get-WindowsFeature FS-Resource-Manager

Display Name                                     Name                               Install State
-----
[X] File Server Resource Manager                FS-Resource-Manager                Installed

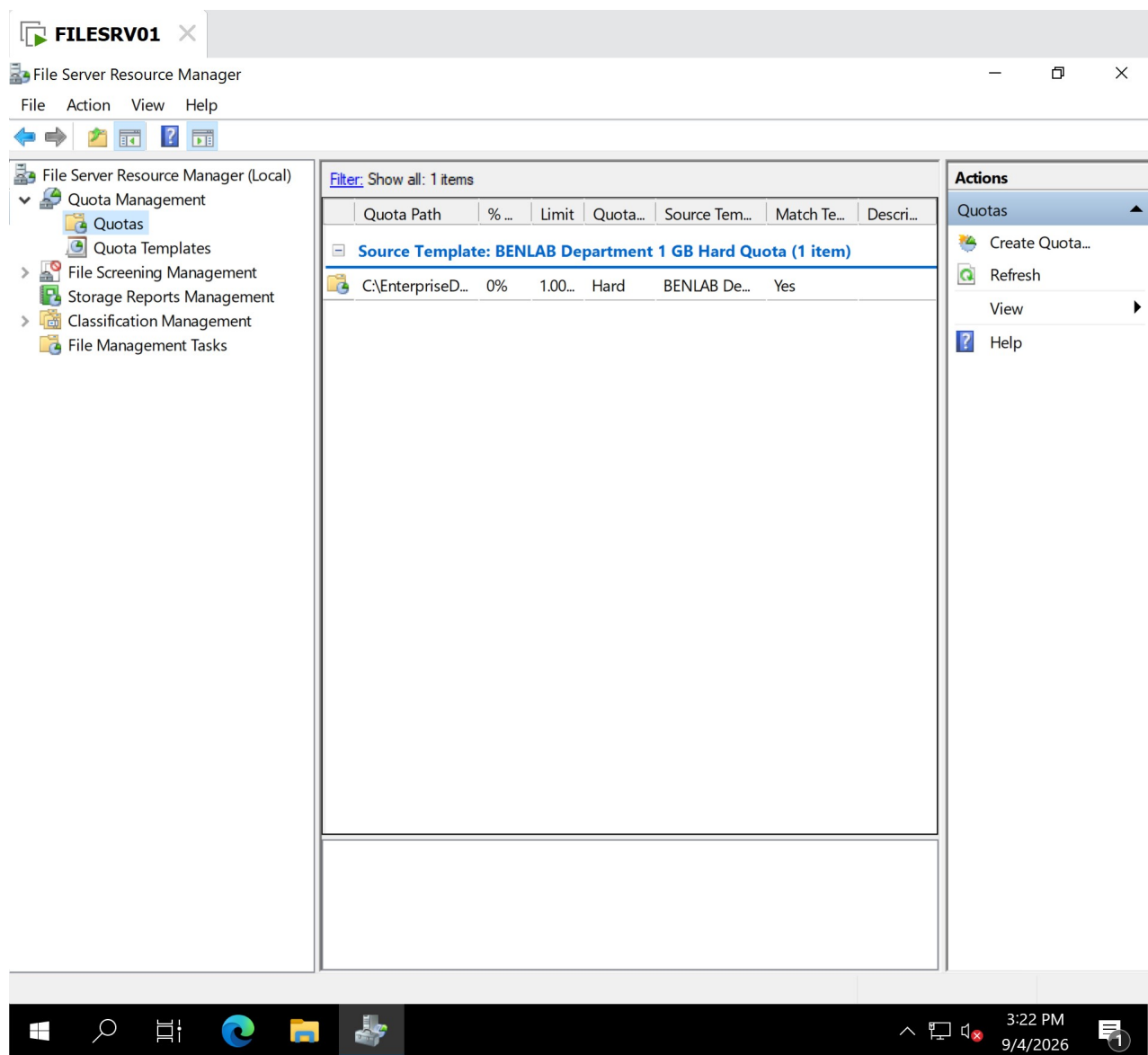
PS C:\Users\Administrator.BENLAB>
Windows Server 2022 Standard Evaluation
Windows License valid for 119 days
Build 20348.fe_release.210507-1500
3:33 PM
9/3/2026

```

Shows Install-WindowsFeature succeeding and Get-WindowsFeature reporting FSRM as **Installed**.



Shows BENLAB Department 1 GB Hard Quota, the **1 GB limit**, **Hard quota**, and the configured **85% Event Log warning**.



Shows the Finance quota listed under **Quota Management** → **Quotas** and associated with the BENLAB template.

The screenshot displays the File Server Resource Manager (FSRM) console window titled "FILESRV01" and a Windows PowerShell window titled "Administrator: Windows PowerShell".

FSRM Console:

- Left Pane:** Shows the navigation tree with "Quota Management" expanded, containing "Quotas" and "Quota Templates".
- Right Pane:** Shows a table of quotas. The filter is "Show all: 1 items". The table has columns: Quota Path, %, Limit, Quota..., Source Tem..., Match Te..., and Descri... The single entry is "Source Template: BENLAB Department 1 GB Hard Quota (1 item)" with a path of "C:\EnterpriseD..." and a limit of "1.00...".
- Actions Pane:** Contains "Create Quota..." and "Refresh".

PowerShell Window:

```

PS C:\Users\Administrator.BENLAB> Get-FsrmQuota -Path "C:\EnterpriseData\Finance"

Description      :
Disabled         : False
MatchesTemplate  : True
Path             : C:\EnterpriseData\Finance
PeakUsage        : 2048
Size             : 1073741824
SoftLimit        : False
Template         : BENLAB Department 1 GB Hard Quota
Threshold        : {MSFT_FSRMQuotaThreshold}
Usage            : 2048
PSComputerName   :

PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB> fsutil file createnew "C:\EnterpriseData\Finance\Quota-Test.bin" 110000000
0
Error: There is not enough space on the disk.
PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB> Test-Path "C:\EnterpriseData\Finance\Quota-Test.bin"
False
PS C:\Users\Administrator.BENLAB>
  
```

The taskbar at the bottom shows the Windows Start button, search icon, task view icon, and several application icons. The system clock indicates 3:42 PM on 9/4/2026.

PowerShell Verification and Enforcement Test

Key Learning Points

- **File Server Resource Manager (FSRM)** provides centralized Windows Server storage-management capabilities beyond ordinary SMB sharing and NTFS permissions.
- A **quota template** allows administrators to standardize storage policies and reuse them across multiple folders.
- A **hard quota** actively prevents storage consumption beyond the configured limit.
- A **soft quota** monitors usage but does not prevent users from exceeding the configured threshold.

- Notification thresholds allow administrators to respond before storage reaches its maximum allocation.
- Windows Event Log integration provides an administrative monitoring mechanism without requiring an email infrastructure in the lab.
- `Get-FsrmQuota` provides a PowerShell method for independently verifying FSRM quota configuration.
- A configuration should not be considered fully validated merely because it appears correctly in the GUI. Testing the actual enforcement behavior provides stronger operational evidence.
- This task followed the administrative workflow:

Configure → Verify → Test → Validate

Task 3 – File Screening Management

Objective

Configure and validate **File Server Resource Manager (FSRM) File Screening** on FILESRV01 to prevent unauthorized executable files from being stored in a departmental data folder while continuing to permit normal business files.

The configuration demonstrates how Windows Server administrators can enforce file-type restrictions and audit policy violations on enterprise file servers.

Environment

Server: FILESRV01

Domain: BENLAB.LOCAL

Role/Feature: File Server Resource Manager (FSRM)

Protected folder:

C:\EnterpriseData\Finance

Screening mode: Active Screening

Blocked file group: Executable Files

Configuration

Step 1 – Review FSRM File Groups

Opened:

File Server Resource Manager → File Screening Management → File Groups

Reviewed the built-in file groups provided by FSRM, including:

- Executable Files
- Audio and Video Files
- Office Files
- Image Files
- Compressed Files
- System Files
- Text Files

The **Executable Files** group was selected for this task because executable content should not normally be stored in the Finance departmental data location.

Step 2 – Create a File Screen Template

Opened:

File Screening Management → File Screen Templates

Created a custom template named:

BENLAB Block Executable Files

Configured:

Screening Type:

Active Screening

Active Screening prevents users from saving files belonging to prohibited file groups rather than merely monitoring the activity.

Selected file group:

Executable Files

Step 3 – Configure Event Log Auditing

Under the template's **Event Log** configuration, enabled:

Send warning to event log

This allows FSRM to generate an administrative event whenever a user attempts to store a prohibited executable file.

The event can record information such as the user, attempted file path, protected location, server, and violated file group.

Step 4 – Apply the File Screen

Created a File Screen on:

C:\EnterpriseData\Finance

Applied the custom template:

BENLAB Block Executable Files

The resulting configuration showed:

Screening Type: Active

File Group: Executable Files

Template: BENLAB Block Executable Files

This established an actively enforced file-type policy on the Finance folder.

Validation

Test 1 – Verify an Allowed File Type

Created a normal text file:

"FSRM normal file test" | Out-File "C:\EnterpriseData\Finance\Allowed-Test.txt"

Verified its existence:

Test-Path "C:\EnterpriseData\Finance\Allowed-Test.txt"

Result:

True

Result: PASS 

The .txt file was successfully created, confirming that the file screen does **not indiscriminately block all writes** to the Finance folder.

Test 2 – Attempt to Store a Prohibited Executable

Attempted to copy Windows Notepad into the protected Finance folder:

```
Copy-Item "C:\Windows\System32\notepad.exe" "C:\EnterpriseData\Finance\notepad-Test.exe"
```

PowerShell returned:

```
Copy-Item : Access to the path  
'C:\EnterpriseData\Finance\notepad-Test.exe' is denied.
```

Result: PASS 

The executable was prevented from being written to the folder, demonstrating successful enforcement of the **Active File Screen**.

Test 3 – Verify the FSRM Audit Event

Opened:

Event Viewer → Windows Logs → Application

Located the FSRM warning generated at the time of the blocked operation.

The event showed:

```
Source: SRMSVC  
Event ID: 8215  
Level: Warning  
Computer: FILESRV01.benlab.local
```

The event reported that:

BENLAB\Administrator

attempted to save:

C:\EnterpriseData\Finance\notepad-Test.exe

and identified the prohibited category as:

Executable Files

Result: PASS 

This confirmed that FSRM not only prevented the prohibited file operation but also **recorded the policy violation for administrative review**.

Final Validation

Task 3 successfully demonstrated the complete FSRM File Screening workflow:

Configure policy → Apply policy → Test allowed file → Test prohibited file → Verify enforcement → Review audit event

The Finance folder continued accepting permitted files while executable files were actively rejected and the attempted violation was recorded in the Windows Application event log.

Key Learning Points

Active vs. Passive Screening: Active Screening actually prevents prohibited files from being saved. Passive Screening allows the files but monitors and reports the activity.

File Groups: FSRM uses file groups to categorize files by filename patterns/extensions, allowing administrators to centrally control categories such as executables, multimedia files, compressed files, and others.

NTFS permissions vs. FSRM: NTFS permissions determine **who can access and modify a folder**, while FSRM File Screening can determine **what types of files may be stored there**. A user may therefore have NTFS write permission but still be prevented from storing a prohibited file type.

Auditing: FSRM can record policy violations in Event Viewer. In this lab, the blocked executable attempt generated **SRMSVC Event ID 8215**, providing evidence of the attempted operation.

Enterprise relevance: File Screening helps administrators enforce storage policies, reduce inappropriate file storage, and provide visibility into violations on centralized file servers.

Screenshots / Evidence

FILESRV01

File Server Resource Manager

File Action View Help

File Server Resource Manager (Local)

- Quota Management
 - Quotas
 - Quota Templates
- File Screening Management
 - File Screens
 - File Screen Templates
 - File Groups
- Storage Reports Management
- Classification Management
- File Management Tasks

File Groups	Include Files	Exclude Files
Audio and...	*.aac, *.aif, *.aiff, *.asf, *.asx, *.au, ...	
Backup Fil...	*.bak, *.bck, *.bkf, *.old	
Compress...	*.ace, *.arc, *.arj, *.bhz, *.bz2, *.ca...	
E-mail Files	*.eml, *.idx, *.mbox, *.mbx, *.ms...	
Executabl...	*.bat, *.cmd, *.com, *.cpl, *.exe, *...	
Image Files	*.bmp, *.dib, *.eps, *.gif, *.img, *...	
Office Files	*.accdb, *.accde, *.accdr, *.accdt, ...	
System Fil...	*.acm, *.dll, *.ocx, *.sys, *.vxd	
Temporar...	*.temp, *.tmp, ~*	
Text Files	*.asc, *.text, *.txt	
Web Page...	*.asp, *.aspx, *.cgi, *.css, *.dhtml, ...	

Actions

File Groups

Create File Group...

Refresh

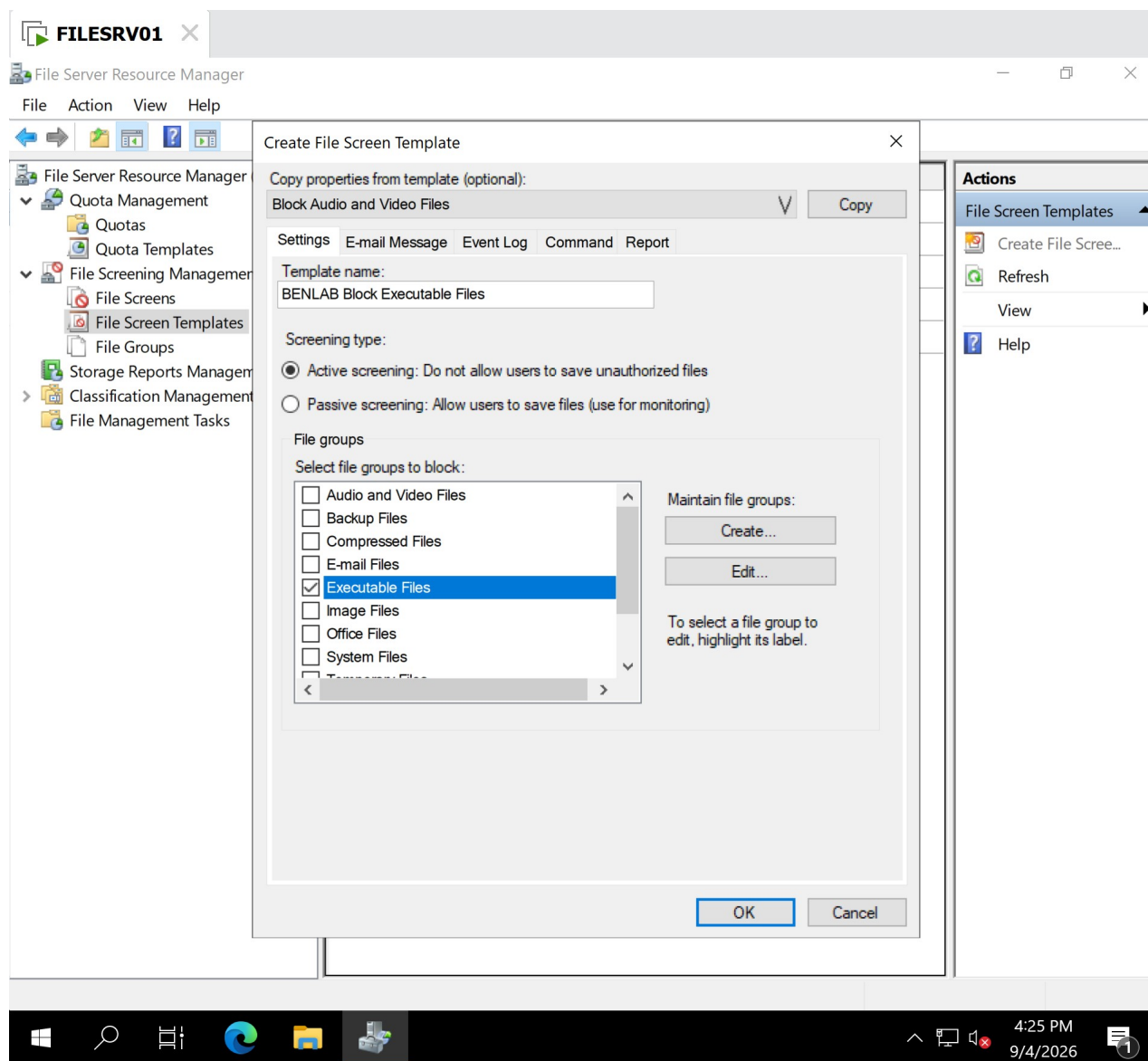
View

Help

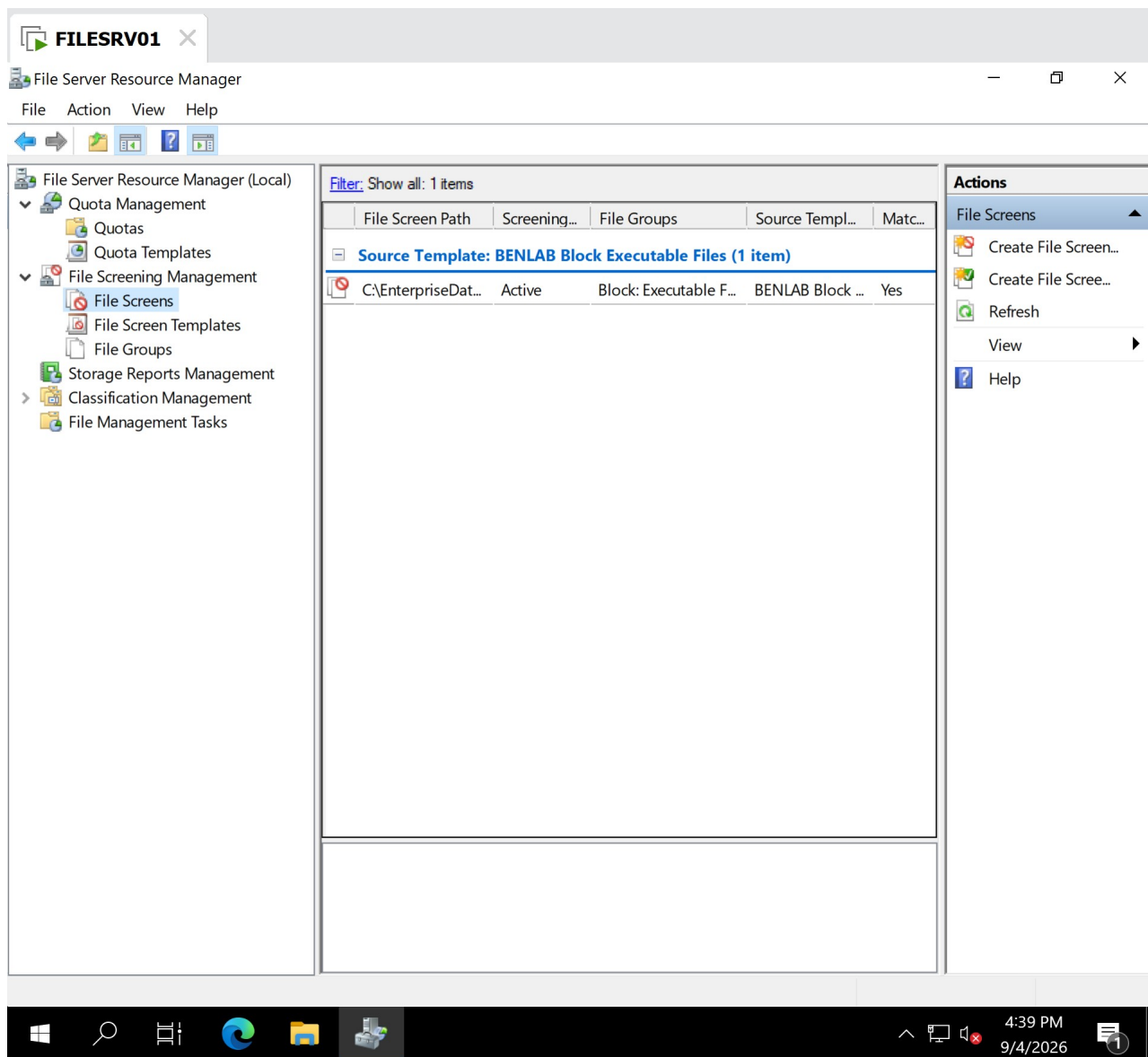
Windows Taskbar

4:13 PM 9/4/2026

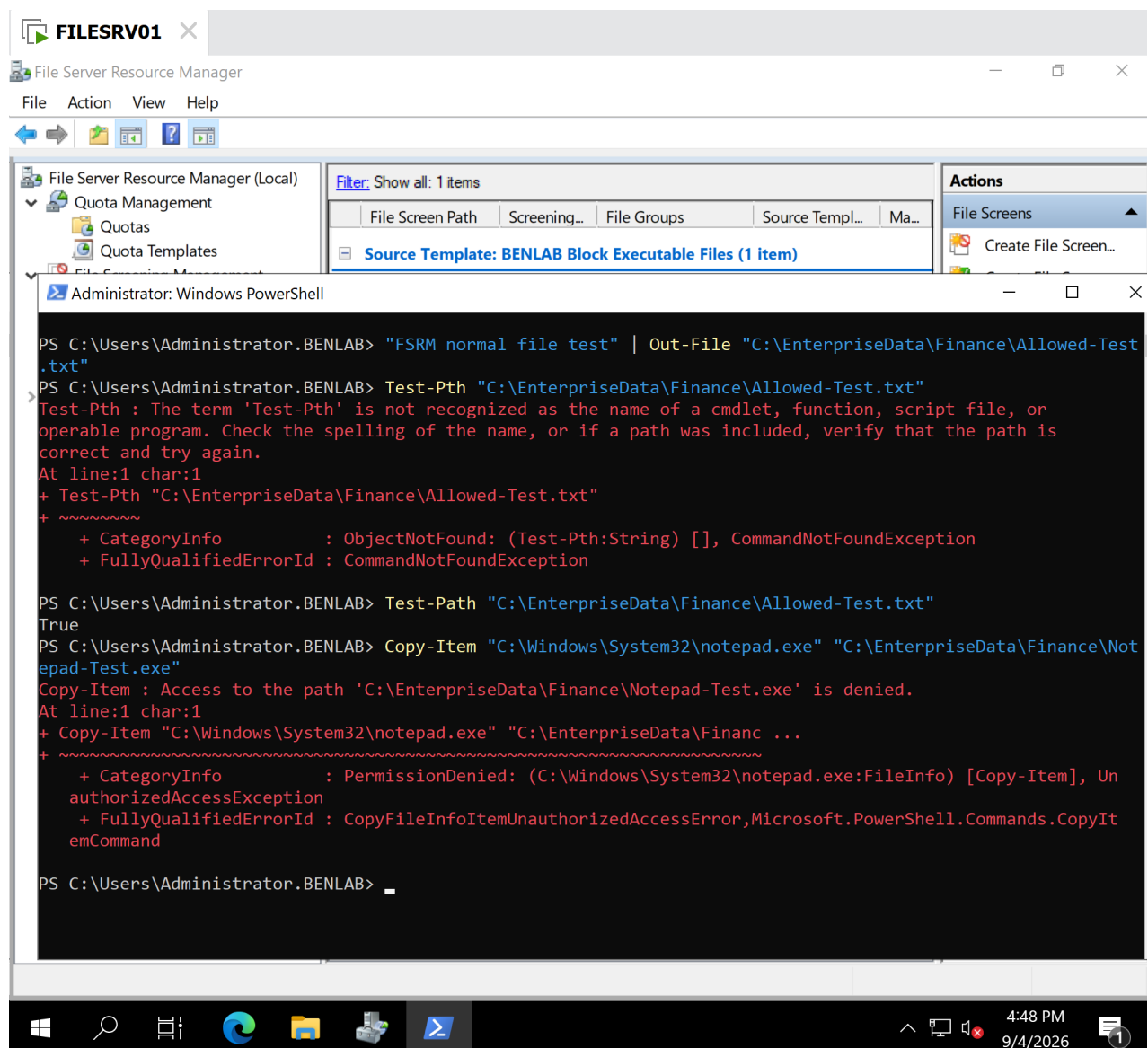
File Groups showing the built-in groups, especially Executable Files.



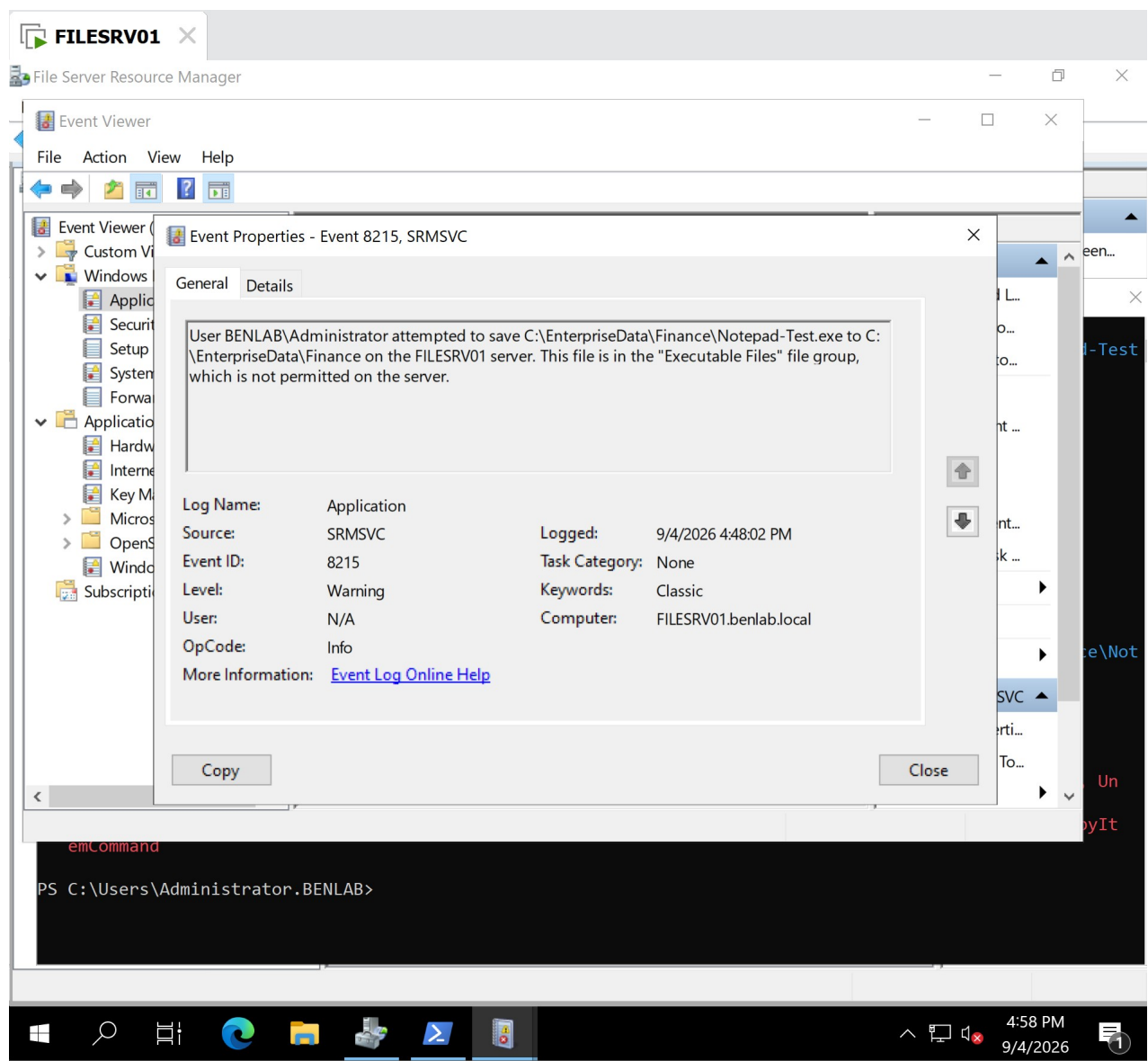
BENLAB Block Executable Files template showing Active Screening + Executable Files.



File Screen on C:\EnterpriseData\Finance showing the BENLAB template applied.



PowerShell validation showing Allowed-Test.txt = True and the attempted .exe returning Access denied.



Event Properties – Event 8215, SRMSVC

Task 4 – Storage Reports Management

Objective

Configure and generate **File Server Resource Manager (FSRM) Storage Reports** on FILESRV01 to analyze data stored within the enterprise file structure.

The task demonstrates how Windows Server administrators can use FSRM reporting to identify file types, file ownership, storage consumption, and potentially large files for storage auditing and capacity management.

Environment

Server: FILESRV01

Domain: BENLAB.LOCAL

Role: File Server

Management Tool: File Server Resource Manager (FSRM)

Report Scope: C:\EnterpriseData

Configuration and Steps

Step 1 – Open Storage Reports Management

On FILESRV01, opened:

File Server Resource Manager → Storage Reports Management

Selected **Generate Reports Now** to create an interactive storage report.

Step 2 – Select Storage Reports

Configured the report to analyze the enterprise data structure.

Selected reports included:

- **Large Files**
- **Files by File Group**

The reports were generated in **DHTML/HTML format** for interactive viewing.

Step 3 – Configure Report Scope

Under the **Scope** tab, added:

C:\EnterpriseData

This allowed FSRM to analyze the enterprise department folders contained beneath this directory, including Finance, Human Resources, Information Technology, and Public.

Step 4 – Configure Report Delivery

Reviewed the **Delivery** configuration.

E-mail delivery was not required for this lab. FSRM was allowed to save the interactive reports locally under:

C:\StorageReports\Interactive

This demonstrated how reports can be retained locally for administrative review without requiring an SMTP configuration.

Validation

Test 1 – Large Files Report

Generated the **Large Files Report** against:

C:\EnterpriseData

The report was configured with a minimum file size of:

5.00 MB

Result

FSRM reported:

0 files meeting the report criteria

This was an expected and valid result because the current lab files within C:\EnterpriseData were smaller than 5 MB.

The result demonstrated that FSRM successfully scanned the specified directory and evaluated its contents against the configured file-size threshold.

Test 2 – Files by File Group Report

Generated the **Files by File Group Report** against the same enterprise data directory.

FSRM successfully detected:

File Groups: 1

Files: 3

Detected File Group: Text Files

The report further analyzed the files by ownership and location.

File Ownership Results

BENLAB\j lee owned **2 files (66.67%)**.

BUILTIN\Administrators owned **1 file (33.33%)**.

Detailed File Analysis

FSRM identified the following test files:

Allowed-Test.txt

Location: C:\EnterpriseData\Finance

Owner: BUILTIN\Administrators

JLee-Finance-Permission-Test.txt.txt

Location: C:\EnterpriseData\Finance

Owner: BENLAB\jlee

JLee-Public-Permission-Test.txt.txt

Location: C:\EnterpriseData\Public

Owner: BENLAB\jlee

All three files were correctly classified within the **Text Files** file group.

Because the files were very small, their reported storage consumption rounded to 0.00 MB.

Results

FSRM Storage Reports successfully:

- Scanned C:\EnterpriseData.
- Evaluated files against a **5 MB large-file threshold**.
- Classified existing files according to file group.
- Identified file ownership.
- Identified the folders containing the files.
- Generated interactive HTML reports for administrator review.
- Stored generated reports under C:\StorageReports\Interactive.

The testing confirmed that FSRM can provide visibility into enterprise file-server storage without modifying or deleting user data.

Key Learning Points

Storage reporting is different from file screening. File screening controls what users are permitted to store, while storage reports provide administrators with visibility into what is already stored.

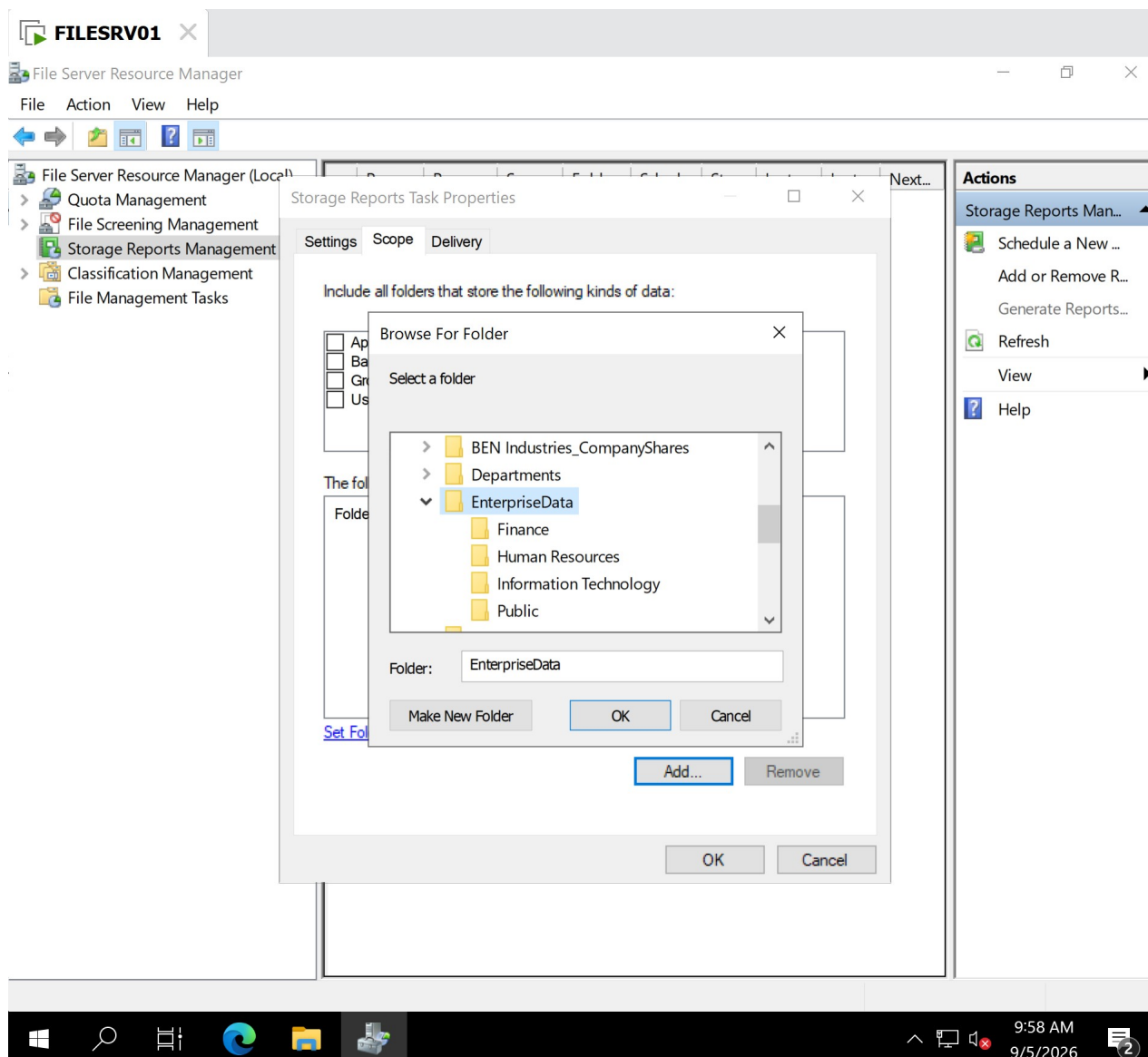
A report containing zero results can still represent a successful scan. The Large Files Report returned zero files because none of the current files met the configured 5 MB threshold.

FSRM can analyze ownership as well as storage consumption. The Files by File Group Report showed that two files belonged to the domain user BENLAB\j lee, while another belonged to the local Administrators group.

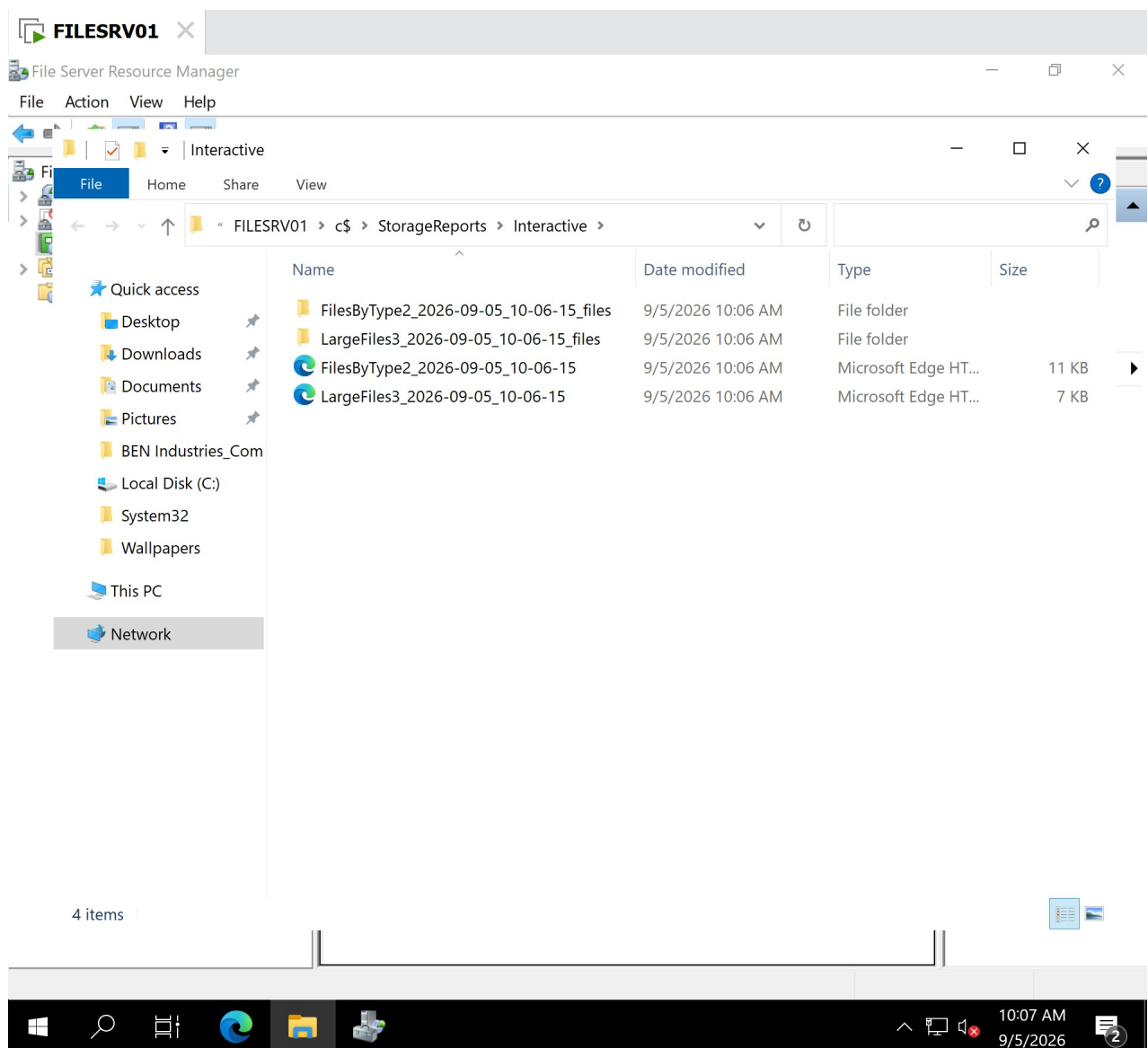
File groups have uses beyond blocking files. The same FSRM classification concepts used during File Screening Management can also help administrators analyze and report on existing data.

Storage reports support proactive administration. In a production environment, administrators can use these reports for capacity planning, storage audits, cleanup decisions, policy development, and identification of unusual storage patterns.

Screenshots / Evidence



FSRM Storage Reports Management configured to generate storage-analysis reports.



FILESRV01

LargeFiles3_2026-09-05_10-06-15

What's new in Microsoft Edge

File

filesrv01/c\$/StorageReports/Interactive/LargeFiles3_2026-09-05_10-06-15.html

Chat

Large Files Report

Generated at: 9/5/2026 10:06:15 AM

Report Description:	Lists files that are a specified size or larger. Use this report to quickly identify the files that are consuming the most disk space on the server. These can help you quickly reclaim large quantities of disk space.		
Machine:	FILESRV01		
Report Folders:	'C:\EnterpriseData'		
Parameters:	Minimum file size: 5.00 MB		

[Large Files Report Table of Contents](#)

[Report Totals](#)
[Size by Owner](#)
[Size by File Group](#)
[Report statistics](#)

Report Totals			
Files shown in the report		All files matching report criteria	
Files	Total size on Disk	Files	Total size on Disk
0	0.00 MB	0	0.00 MB

[To top of the current report](#)

Size by Owner		
Owner	Total size on Disk	Files

[To top of the current report](#)

Size by File Group		
File Group	Total size on Disk	Files

[To top of the current report](#)

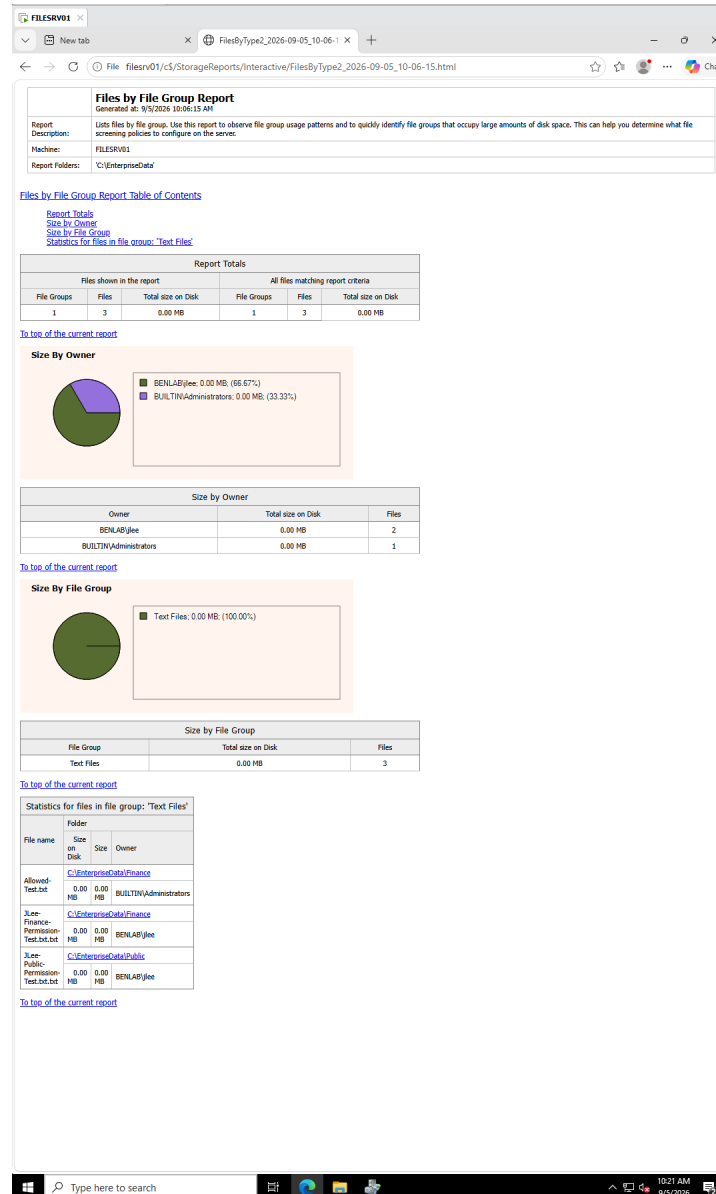
Report statistics		
-------------------	--	--

10:09 AM

9/5/2026

2

Large Files Report showing a **5.00 MB** minimum file size and **0 files** matching the criteria.



Report showing:

- C:\EnterpriseData as the report scope
- 3 detected files
- Text Files classification
- BENLAB\jlee ownership of two files
- BUILTIN\Administrators ownership of one file
- Individual filenames and folder locations

Task 5 – Configure and Validate FSRM File Management Tasks

Objective

Configure a File Server Resource Manager (FSRM) File Management Task on FILESRV01 to identify files that have not been modified for more than 30 days and automatically move qualifying files to a designated expiration directory. Validate the configuration using a controlled test file.

Configuration

Server: FILESRV01

File Management Task: BENLAB Stale File Management

Scope:

C:\EnterpriseData\FileManagementTest

Condition:

Days since file was last modified: 30

This condition identifies files whose last modification time exceeds the 30-day threshold.

Action:

File expiration

Expiration directory:

C:\EnterpriseData\ExpiredFiles

The task was configured with a weekly schedule and enabled for automatic execution.

Validation Procedure

A controlled test file was created in the File Management Task scope:

```
New-Item "C:\EnterpriseData\FileManagementTest\StaleFile-Test.text.txt" -ItemType File
```

Because a newly created file would not meet the 30-day condition, its `LastWriteTime` was modified to simulate a stale file:

```
(Get-Item "C:\EnterpriseData\FileManagementTest\StaleFile-Test.text.txt").LastWriteTime = (Get-Date).AddDays(-31)
```

The timestamp was then verified:

```
Get-Item "C:\EnterpriseData\FileManagementTest\StaleFile-Test.text.txt" |
```

Select-Object Name, LastWriteTime

Observed result:

StaleFile-Test.text.txt 8/6/2026 9:34:06 AM

This confirmed that the test file exceeded the configured 30-day stale-file threshold.

Manual Task Execution

Rather than waiting for the scheduled execution, the File Management Task was manually initiated through:

File Server Resource Manager

→ File Management Tasks

→ BENLAB Stale File Management

→ Run File Management Task Now

The option **Wait for the task to complete** was selected to observe the test execution directly.

FSRM Report Validation

After execution, FSRM generated a **File Management Task Report**.

The report confirmed:

- Task: BENLAB Stale File Management
- Machine: FILESRV01
- Action type: **Expiration**
- Scope: C:\EnterpriseData\FileManagementTest
- Files matching report criteria: 1
- Matching file: StaleFile-Test.text.txt
- Last modified: 8/6/2026 9:34:06 AM
- No file-processing error was reported.

This demonstrated that FSRM correctly evaluated the configured condition and identified the intentionally aged test file.

Expiration Verification

PowerShell was used to verify the contents of the expiration directory:

```
Get-ChildItem "C:\EnterpriseData\ExpiredFiles" -Recurse
```

The expired file was located under:

C:\EnterpriseData\ExpiredFiles

```
└─ FILESRV01
    └─ 2026-09-06_09-44-06
        └─ C$
            └─ EnterpriseData
                └─ FileManagementTest
                    └─ StaleFile-Test.text.txt
```

This confirmed that FSRM successfully moved the qualifying file from its original location into the configured expiration directory while preserving information about its original path.

Result

Task 5 completed successfully.

FSRM was configured and validated to automatically manage stale data based on file age. A controlled file was artificially aged beyond the 30-day threshold, detected by the File Management Task, processed through the configured expiration action, and successfully located in the expiration directory.

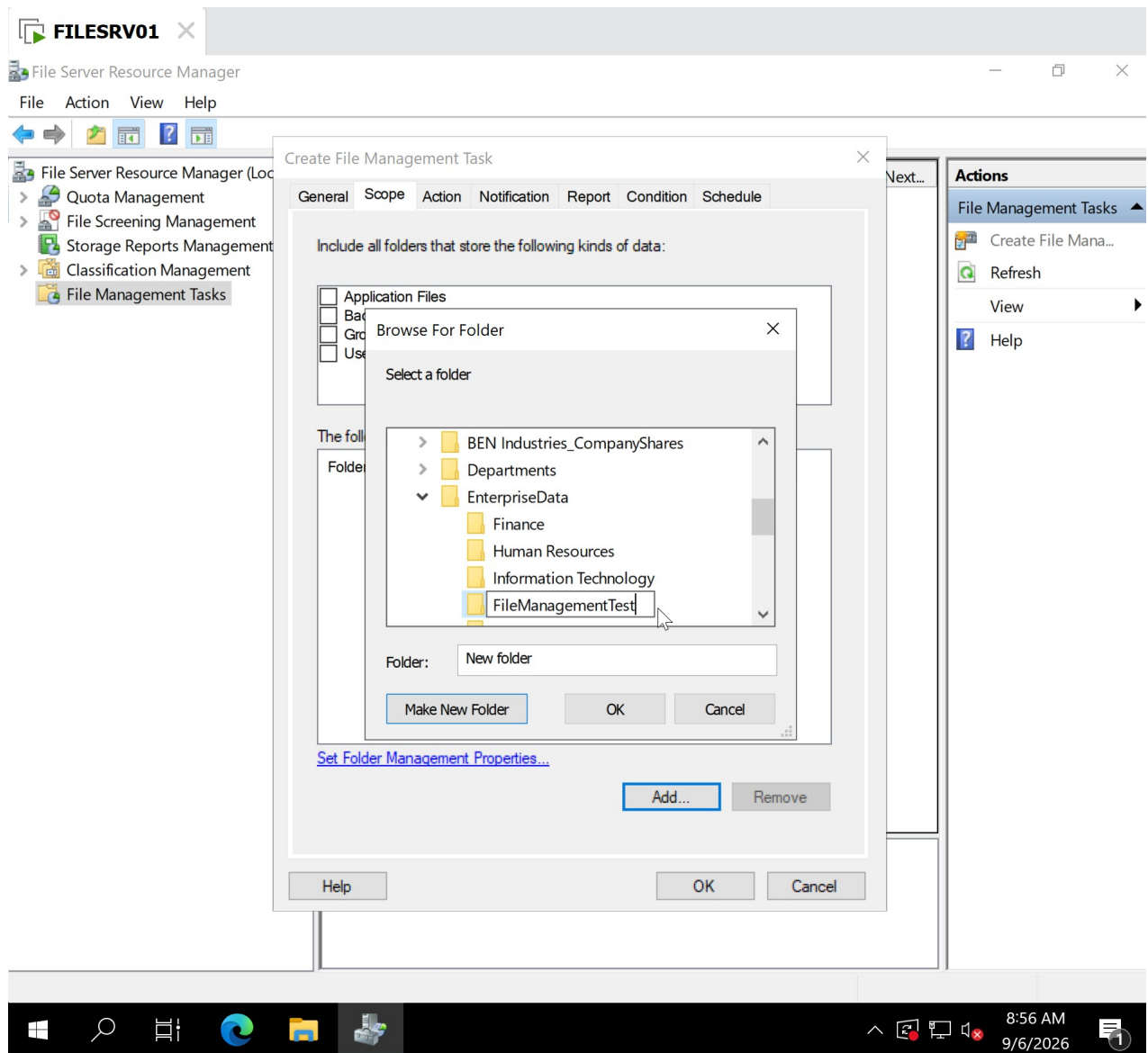
The validation demonstrated an end-to-end file lifecycle workflow:

Identify → Evaluate condition → Expire → Relocate → Verify

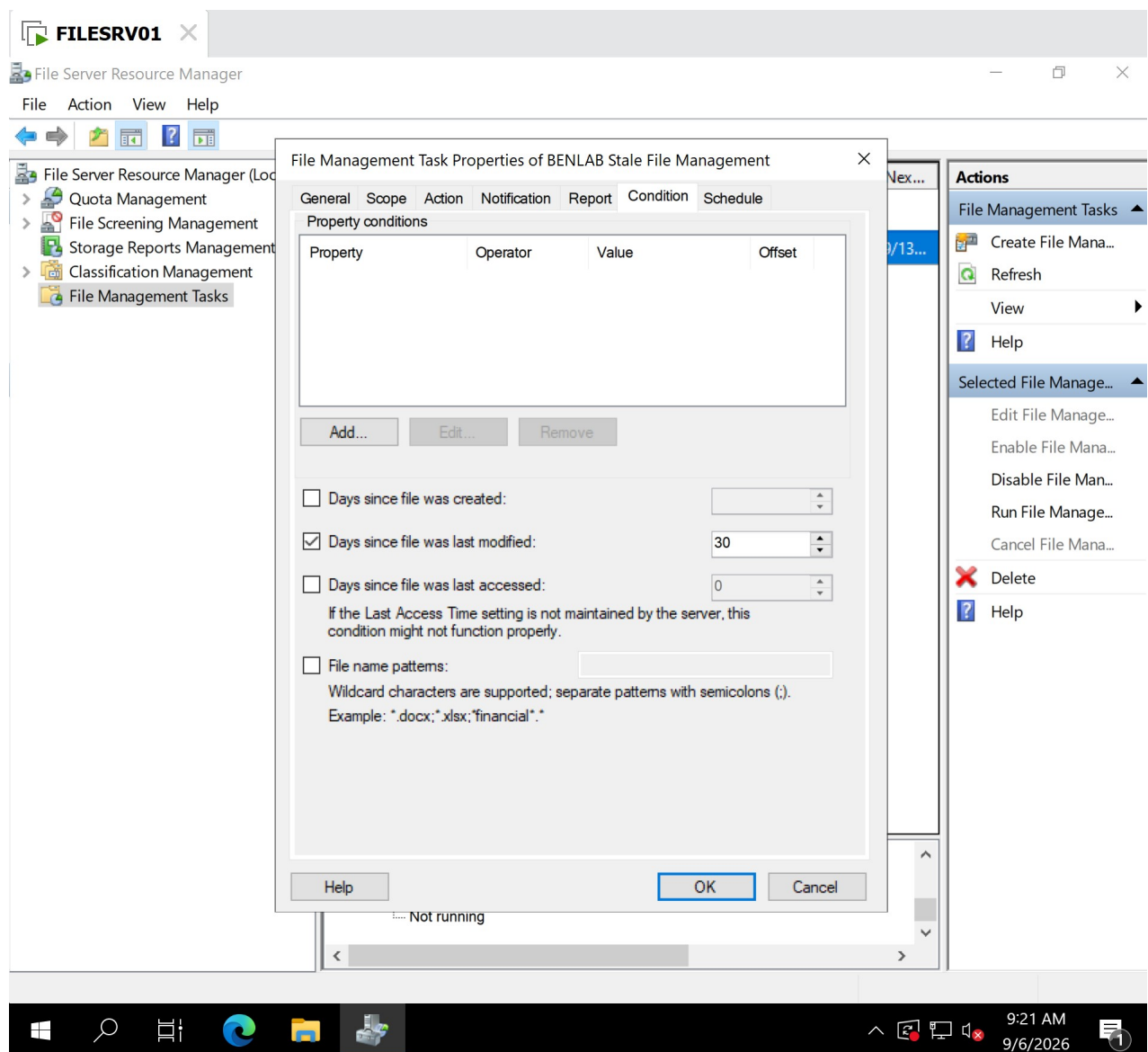
Key Learning Points

- **File Management Tasks** automate file lifecycle operations based on defined conditions.
 - File age can be evaluated using attributes such as **last modified time**.
 - **File expiration** does not necessarily mean immediate deletion; the configured action can relocate stale files to a controlled location.
 - FSRM creates reports that provide evidence of which files matched the configured criteria.
 - PowerShell provides an effective method for creating controlled test conditions and independently verifying FSRM results.
 - Preserving the original directory hierarchy in the expiration location helps administrators determine where expired files originated.
 - Production file-management policies should be tested against controlled data before being applied broadly to business shares.
-

Screenshots / Evidence



FSRM File Management Task configuration showing the scope/action.



Condition tab showing Days since file was last modified: 30.

FSRM File Management Task Report showing the one matching file.

FILESRV01

FileManagement-BENLAB State Fi

FileManagement-BENLAB State File Management Task Report

filesrv01/CS/StorageReports/Interactive/FileManagement-BENLAB%20State%20File%20Management_2026-09-06...

File Management Task Report

Generated at: 9/6/2026 9:44:06 AM

Report Description: Report for the following File Management Task: BENLAB State File Management

Action Type: expiration - Expiration directory C:\EnterpriseData\EnterpriseFiles\FILESRV01\2026-09-06_09-44-06

Machine: FILESRV01

Report Folder: C:\EnterpriseData\FileManagementTest

[File Management Task Report Table of Contents](#)
[Report Totals](#)
[Size by Owner](#)
[Size by File Group](#)
[Report statistics](#)
[Report Error for Files](#)

Report Totals			
Files shown in the report		All files matching report criteria	
Files	Total size on Disk	Files	Total size on Disk
1	0.00 MB	1	0.00 MB

[To top of the current report](#)

Size By Owner

BUILTIN\Administrators: 0.00 MB (100.00%)

Size by Owner		
Owner	Total size on Disk	Files
BUILTIN\Administrators	0.00 MB	1

[To top of the current report](#)

Size By File Group

Text Files: 0.00 MB (100.00%)

Size by File Group		
File Group	Total size on Disk	Files
Text Files	0.00 MB	1

[To top of the current report](#)

Report statistics						
File name	Folder	Size on Disk	Size	Created	Last accessed	Last modified
	Owner					
StateFile-Test.txt	C:\EnterpriseData\FileManagementTest					
	BUILTIN\Administrators	0.00 MB	0.00 MB	9/6/2026 9:28:17 AM	9/6/2026 9:28:17 AM	9/6/2026 9:34:06 AM

[To top of the current report](#)

Error for files

File name	Folder	Error	Owner
-----------	--------	-------	-------

[To top of the current report](#)

Type here to search

Network 2 Internet access

Final PowerShell
verification showing
StaleFile-
Test.text.txt
underneath\Enterpr
iseData\ExpiredF
iles.

```
FILESRV01
Administrator Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Shell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\Administrator.BENLAB> New-Item "C:\EnterpriseData\FileManagementTest\StaleFile-Test.text.txt" -ItemType File
PS C:\Users\Administrator.BENLAB>
Directory: C:\EnterpriseData\FileManagementTest

Mode                LastWriteTime         Length Name
----                -
-a-----          9/6/2026   9:28 AM              0 StaleFile-Test.text.txt

PS C:\Users\Administrator.BENLAB> (Get-Item "C:\EnterpriseData\FileManagementTest\StaleFile-Test.text.txt").LastWriteTime - (Get-Date).AddDays(-31)
PS C:\Users\Administrator.BENLAB>
PS C:\Users\Administrator.BENLAB> Get-Item "C:\EnterpriseData\FileManagementTest\StaleFile-Test.text.txt" | Select-Object Name, LastWriteTime

Name                LastWriteTime
-----
StaleFile-Test.text.txt 8/6/2026 9:34:06 AM

PS C:\Users\Administrator.BENLAB> Get-ChildItem "C:\EnterpriseData\ExpiredFiles" -Recurse

Directory: C:\EnterpriseData\ExpiredFiles

Mode                LastWriteTime         Length Name
----                -
j-----          9/6/2026   9:44 AM              0 FILESRV01

Directory: C:\EnterpriseData\ExpiredFiles\FILESRV01

Mode                LastWriteTime         Length Name
----                -
j-----          9/6/2026   9:44 AM              0 2026-09-06_09-44-06

Directory: C:\EnterpriseData\ExpiredFiles\FILESRV01\2026-09-06_09-44-06

Mode                LastWriteTime         Length Name
----                -
j-----          9/6/2026   9:44 AM              0 CS
-a-----          9/6/2026   9:44 AM          304 Readme.txt

Directory: C:\EnterpriseData\ExpiredFiles\FILESRV01\2026-09-06_09-44-06\CS

Mode                LastWriteTime         Length Name
----                -
j-----          9/6/2026   9:44 AM              0 EnterpriseData

Directory: C:\EnterpriseData\ExpiredFiles\FILESRV01\2026-09-06_09-44-06\CS\EnterpriseData

Mode                LastWriteTime         Length Name
----                -
j-----          9/6/2026   9:44 AM              0 FileManagementTest

Directory: C:\EnterpriseData\ExpiredFiles\FILESRV01\2026-09-06_09-44-06\CS\EnterpriseData\FileManagementTest

Mode                LastWriteTime         Length Name
----                -
-a-----          8/6/2026   9:34 AM              0 StaleFile-Test.text.txt

PS C:\Users\Administrator.BENLAB>
```

Task 6 – Configure and Validate Shadow Copies

Objective

Configure **Volume Shadow Copy Service (VSS)** on FILESRV01 to provide point-in-time recovery of files stored on the server.

The task also validates a realistic recovery scenario in which a file is accidentally overwritten and an administrator restores an earlier version without restoring the entire server from backup.

Environment

Server: FILESRV01

Domain: BENLAB.LOCAL

Operating System: Windows Server 2022

Volume: C:\

Test folder:

C:\EnterpriseData\FileManagementTest

Recovery test file:

ShadowCopy-Recovery-Test.txt

Step 1 – Verify the Server Volume

PowerShell was used to inspect the available volumes:

```
Get-Volume
```

The command confirmed that the C: volume uses **NTFS**, is healthy, and had sufficient free disk space for Shadow Copy storage.

Key learning point: Shadow Copies operate at the **volume level**. Enabling Shadow Copies on C: therefore protects eligible data stored on that volume rather than enabling the feature separately on each individual folder.

Step 2 – Configure Shadow Copy Storage

From the properties of the C: drive:

Properties → Shadow Copies → C: → Settings

Shadow Copy storage was configured on the C: volume with a storage limit of approximately:

8127 MB

This prevents VSS snapshots from consuming unlimited disk space.

Key learning point: Shadow Copies require disk space to maintain historical block changes. When the allocated storage becomes full, Windows can remove older shadow copies to make room for newer ones.

Step 3 – Configure the Shadow Copy Schedule

Two scheduled Shadow Copy creation times were configured:

7:00 AM – Monday through Friday

12:00 PM – Monday through Friday

This provides multiple recovery points during a normal business day.

For example:

7:00 AM Snapshot

↓

User modifies files

↓

12:00 PM Snapshot

↓

User modifies files

If a file is accidentally changed during the morning, the administrator may be able to recover the version captured by the **7:00 AM snapshot**.

Important: Shadow Copies do not continuously save every change. Recovery is based on the versions available at the configured snapshot points.

Step 4 – Enable Shadow Copies

Shadow Copies were enabled for the C : volume.

The Shadow Copies interface subsequently displayed:

- Shadow Copies enabled
- Scheduled next run
- Storage consumption on C :
- Existing shadow-copy recovery points

This confirmed that VSS was operational.

Step 5 – Create a Manual Recovery Point

A manual Shadow Copy was created using:

Create Now

The interface displayed a new recovery point, providing a controlled snapshot for the recovery test.

A second recovery point was later visible as well, confirming that multiple point-in-time versions could be maintained.

Step 6 – Create the Recovery Test File

The following file was created inside the test directory:

C:\EnterpriseData\FileManagementTest\ShadowCopy-Recovery-Test.txt

The original contents were:

BENLAB Finance Report - ORIGINAL VERSION

A Shadow Copy was captured while this original version existed.

Step 7 – Simulate an Accidental File Overwrite

To reproduce a realistic end-user mistake, the contents of the file were changed to:

BENLAB Finance Report - ACCIDENTALLY OVERWRITTEN

The modified file was saved.

This simulated a situation in which an employee unintentionally replaces valid business information.

Step 8 – Locate the Previous Version

The file properties were opened:

Right-click file → Properties → Previous Versions

Windows displayed a previous version from the Shadow Copy.

The administrator selected the historical version and used **Open** first to inspect it.

The historical copy contained:

BENLAB Finance Report - ORIGINAL VERSION

This confirmed that the desired data existed inside the Shadow Copy before performing a restore.

Key learning point: When possible, inspecting a previous version before restoring it is safer than immediately replacing the production copy.

Step 9 – Restore the Previous Version

After verifying the historical copy, **Restore** was selected.

Windows displayed a warning explaining that restoring the previous version would replace the current version and that the operation could not simply be undone.

The restoration was then confirmed.

Step 10 – Verify Successful Recovery

The live file was reopened after restoration.

Its contents were successfully returned to:

BENLAB Finance Report - ORIGINAL VERSION

The file size also returned from approximately **48 bytes** for the overwritten version to approximately **40 bytes** after restoration.

This provided direct evidence that the historical version replaced the accidentally modified version.

Validation Results

Test	Result
NTFS volume verified	✓ Passed
Shadow Copy storage configured	✓ Passed
Shadow Copies enabled	✓ Passed
7:00 AM schedule configured	✓ Passed
12:00 PM schedule configured	✓ Passed
Manual recovery point created	✓ Passed
Original test file captured	✓ Passed
Accidental overwrite simulated	✓ Passed
Previous version located	✓ Passed
Historical contents inspected	✓ Passed
Previous version restored	✓ Passed
Original contents recovered	✓ Passed

Key Learning Points

Shadow Copies are point-in-time recovery, not traditional backups. They are extremely useful for quickly recovering files that users accidentally delete, overwrite, or modify, but they should complement—not replace—a proper backup strategy.

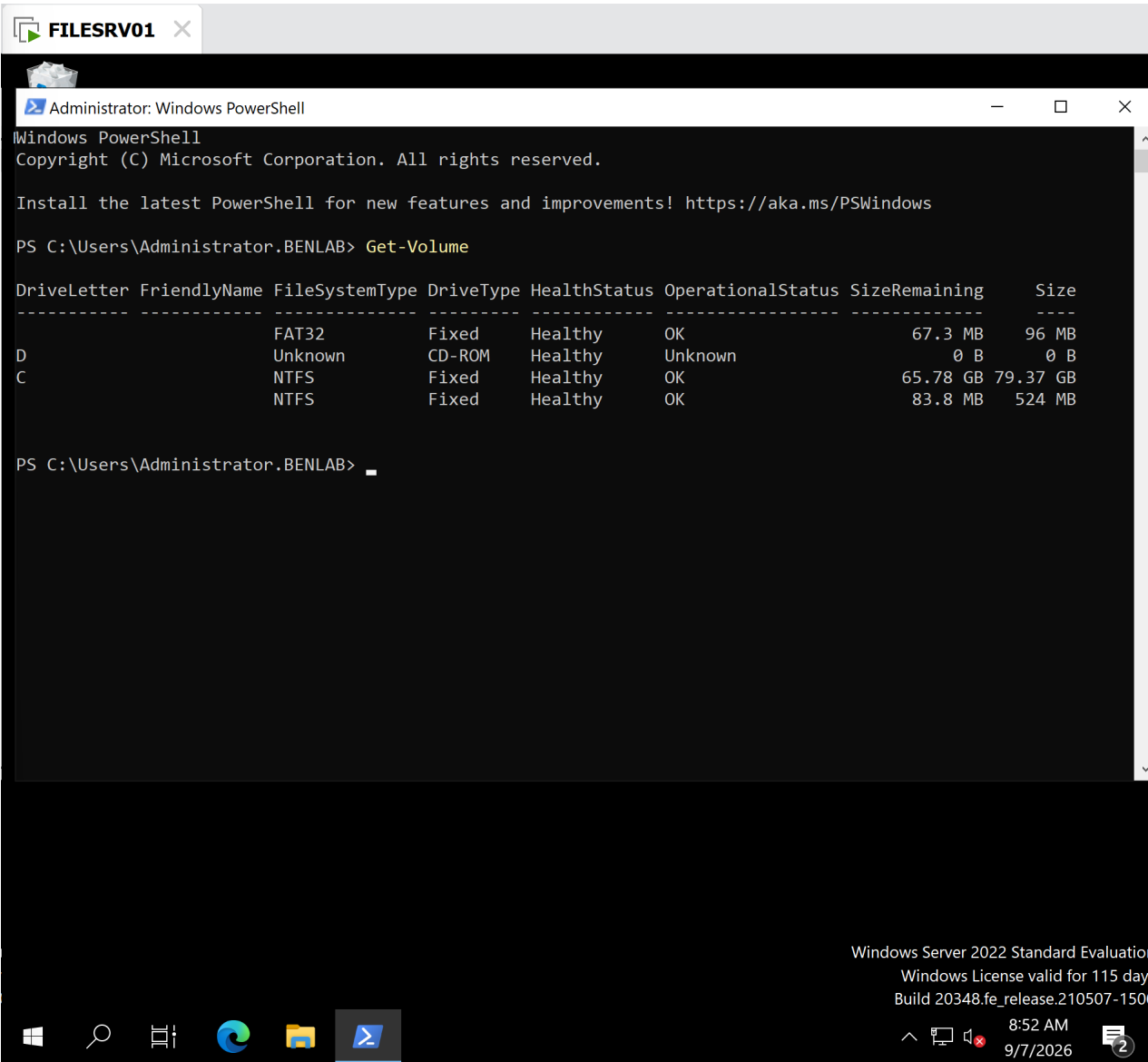
The administrator can use **Previous Versions** to recover individual files without restoring an entire server or volume.

The lab also demonstrated why snapshot frequency matters. With recovery points at **7:00 AM and 12:00 PM**, the available recovery version depends on when the last Shadow Copy was created relative to the user's mistake.

Most importantly, this task demonstrated the complete administrative workflow:

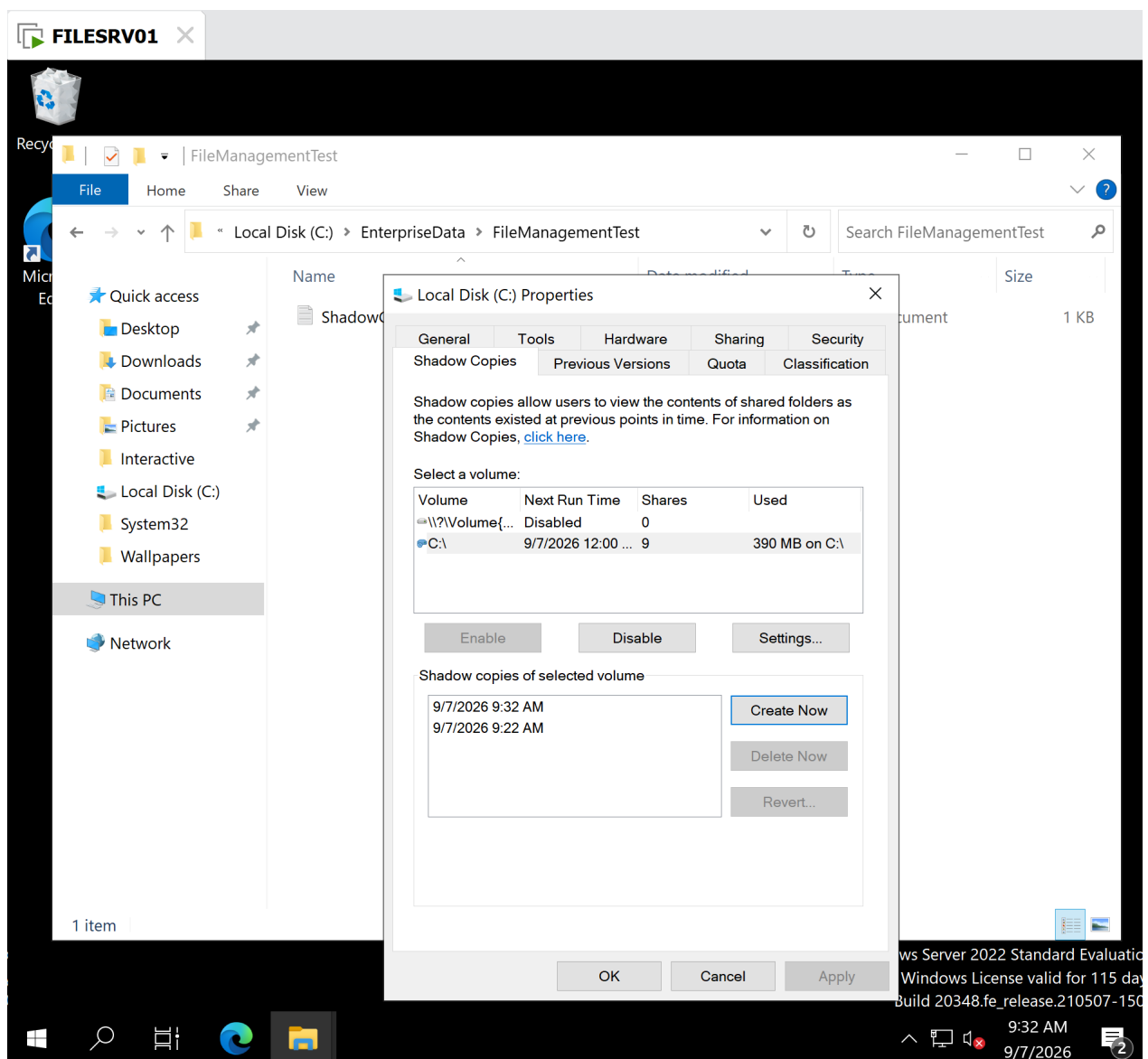
Configure → Schedule → Capture → Simulate Failure → Locate Previous Version → Inspect → Restore → Verify

Screenshots /Evidence



Evidence: Get -Vo l u m e output showing the C : volume as NTFS, healthy, and with sufficient free space.

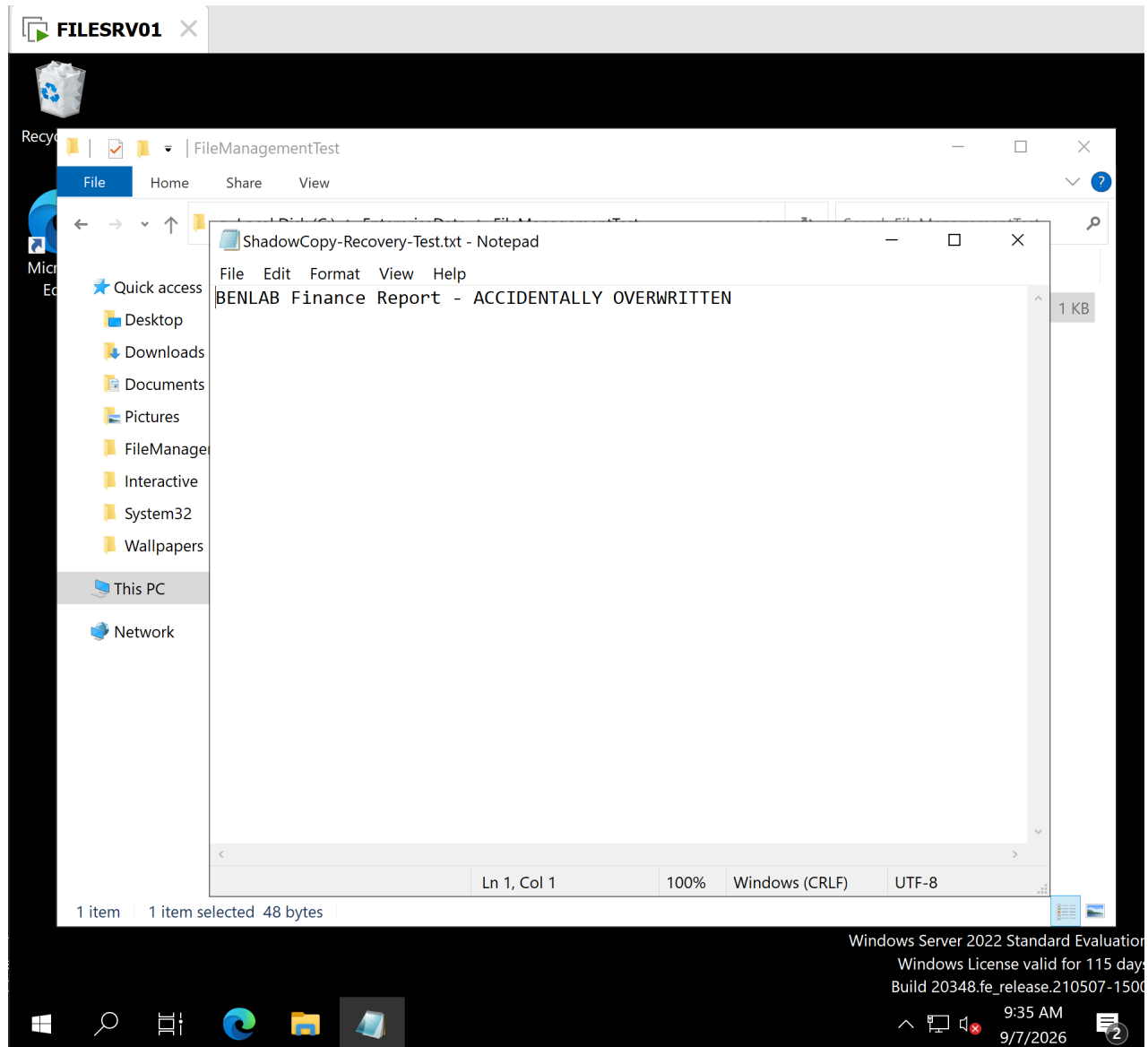
Purpose: Confirms the storage volume being protected by Shadow Copies and validates available capacity before configuration.



Evidence: Shadow Copies tab showing:

- Shadow Copies enabled
- 9/7/2026 9:22 AM
- 9/7/2026 9:32 AM
- VSS storage usage on C :

Purpose: Confirms that VSS is operational and that point-in-time recovery snapshots were successfully created.

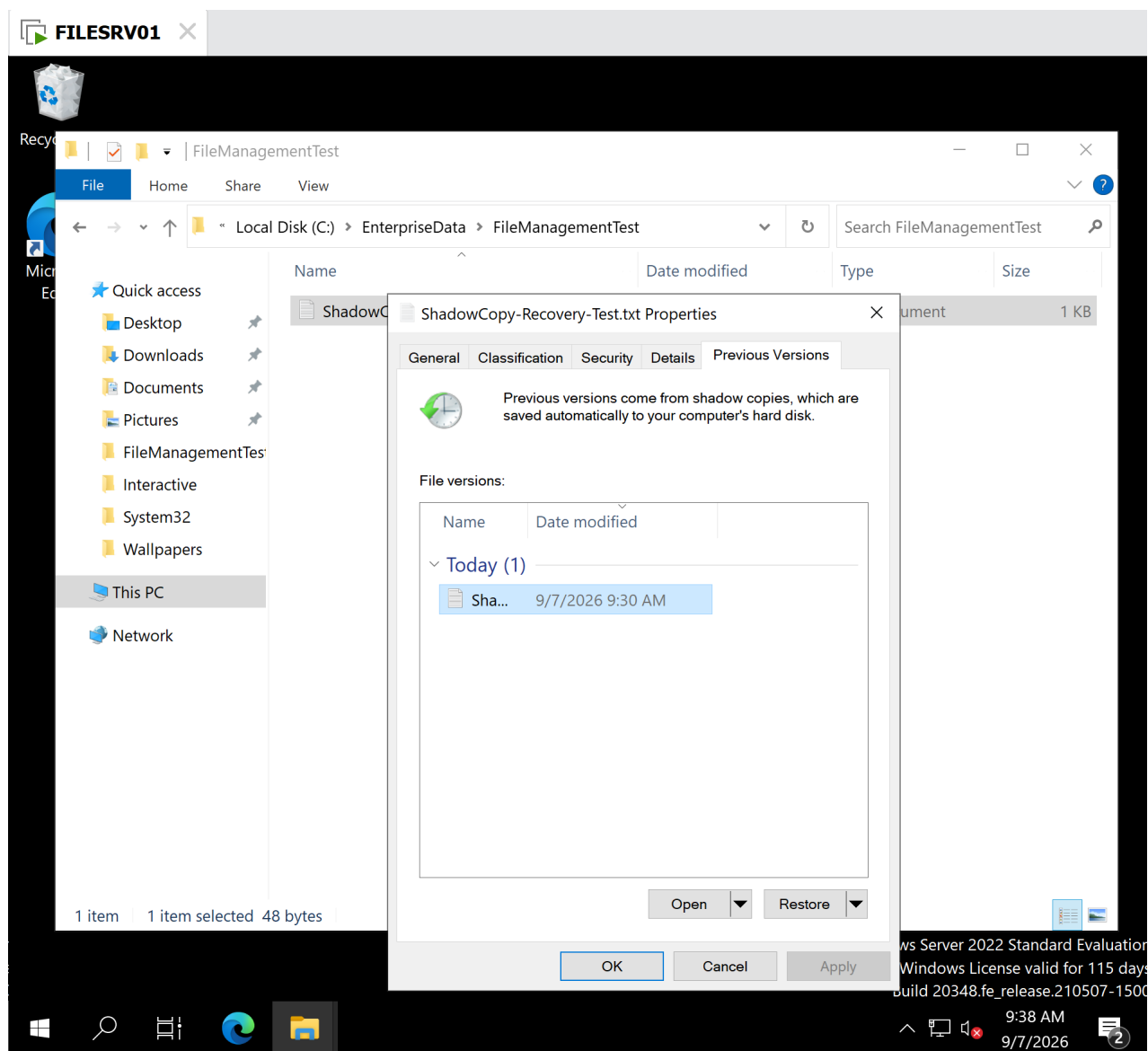


Simulated Accidental Overwrite

Evidence: Same file containing:

BENLAB Finance Report - ACCIDENTALLY OVERWRITTEN

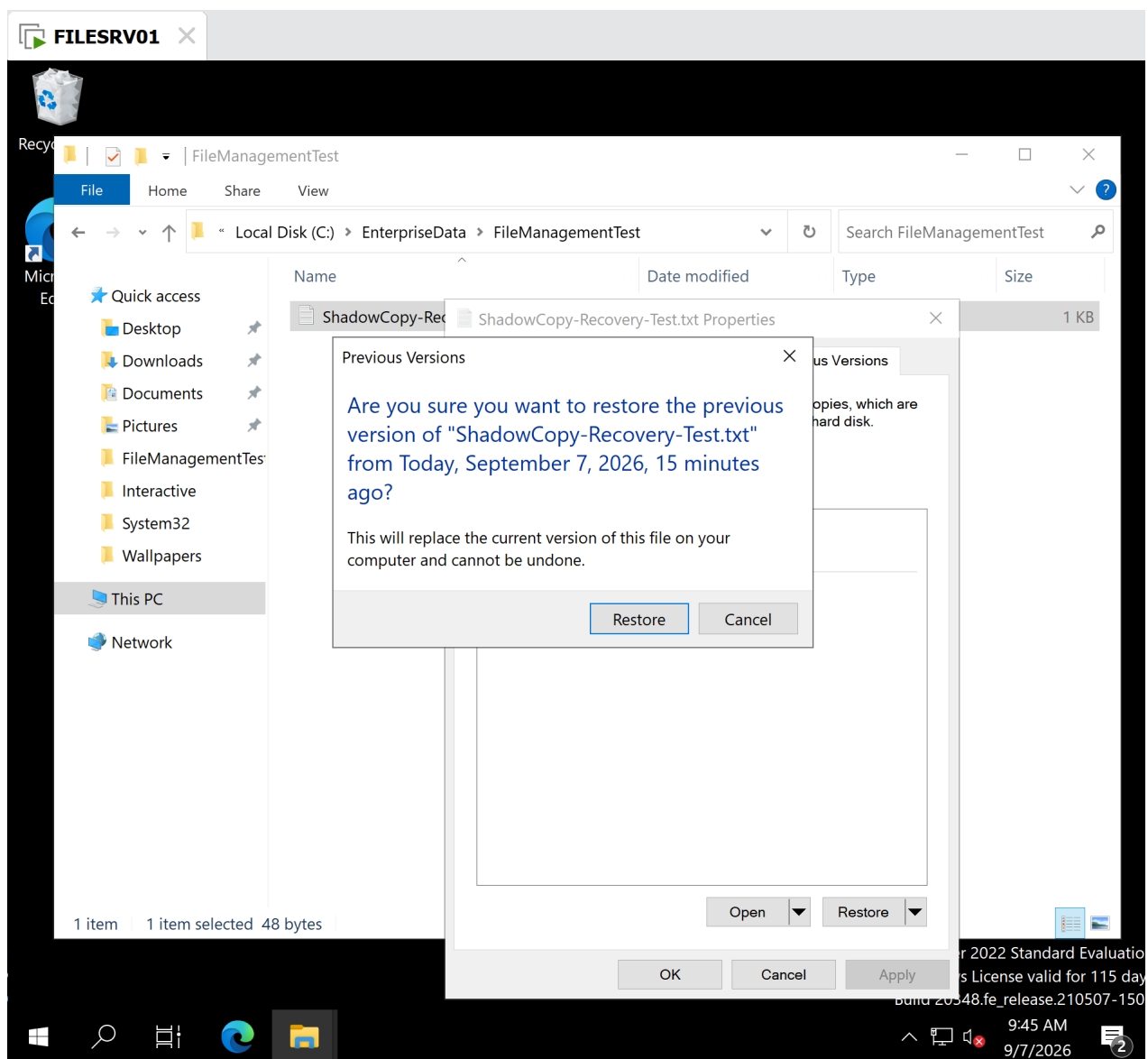
Purpose: Documents the simulated end-user incident requiring recovery.



Evidence: Historical copy opened from **Previous Versions**, displaying:

BENLAB Finance Report - ORIGINAL VERSION

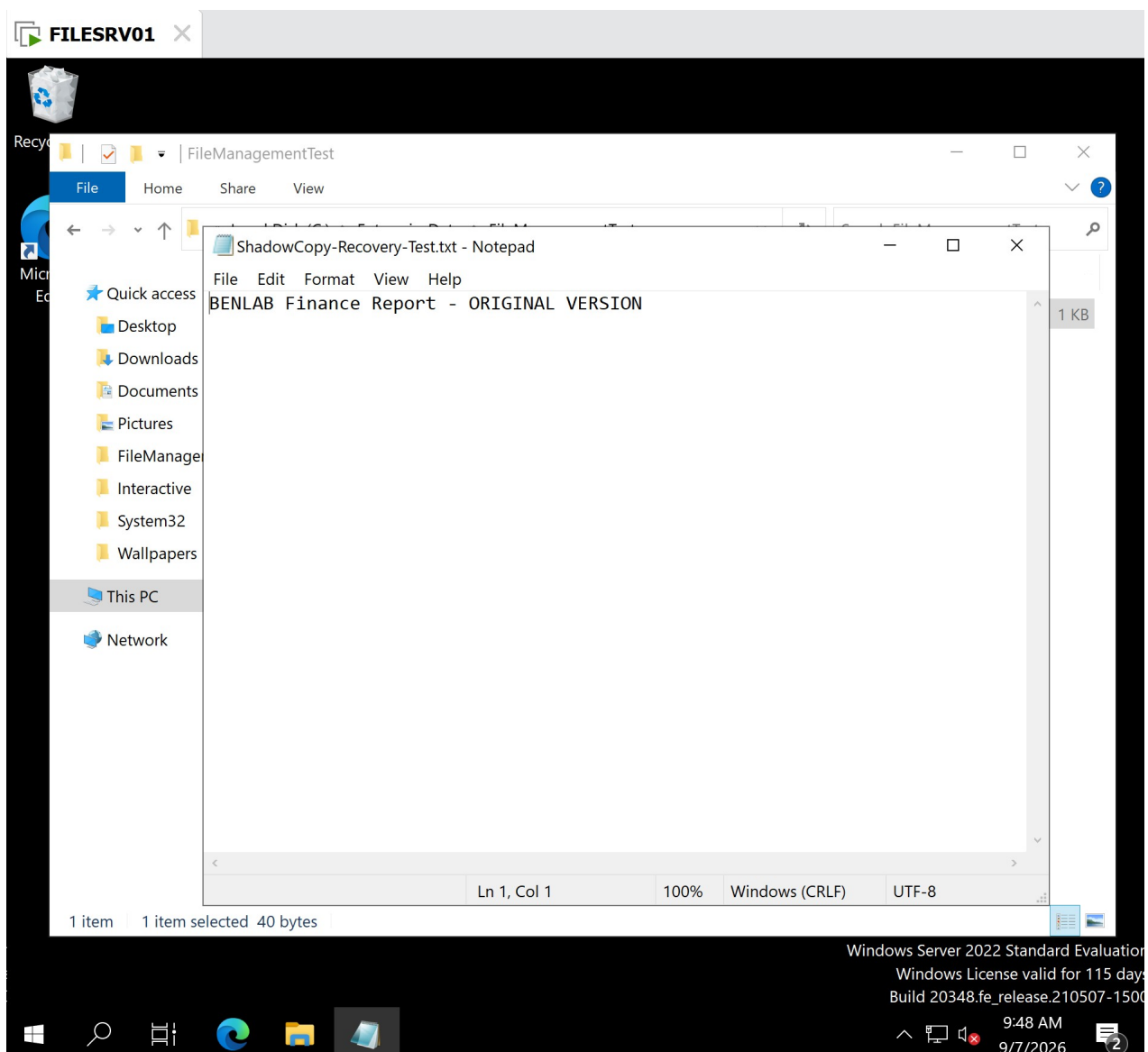
Purpose: Demonstrates the safer administrative practice of inspecting the historical version before restoring it.



Evidence: Windows warning:

Restoring the previous version will replace the current version and cannot be undone.

Purpose: Documents the administrative confirmation step before overwriting the live file.



Evidence: Live ShadowCopy-Recovery-Test . txt reopened after restoration and displaying:
BENLAB Finance Report - ORIGINAL VERSION

Purpose: Final proof that the previous version was successfully restored.

Task 7 — Configure and Validate a Domain-Based DFS Namespace

Objective

Configure a **domain-based Distributed File System (DFS) Namespace** on FILESRV01 to provide users with a centralized, logical path for accessing departmental file shares without requiring users to know the physical file server hosting the data.

The namespace was designed to provide access through:

`\\benlab.local\CorporateData`

rather than requiring users to browse directly to:

`\\FILESRV01\CompanyShares`

Environment

Domain: benlab.local

Namespace Server: FILESRV01

DFS Namespace: \\benlab.local\CorporateData

Existing SMB Share: \\FILESRV01\CompanyShares

Validation Workstation: ADMINPC01

Step 1 — Open DFS Management

Opened **DFS Management** on FILESRV01 and selected:

Namespaces → New Namespace

FILESRV01 was selected as the namespace server.

Step 2 — Create the CorporateData Namespace

Created a new namespace named:

CorporateData

A **domain-based namespace** was selected, resulting in the UNC path:

`\\benlab.local\CorporateData`

Windows Server 2008 mode was enabled.

The namespace root was created locally under:

C:\DFSRoots\CorporateData

The New Namespace Wizard completed successfully.

Step 3 — Validate the Existing SMB Share

Before creating DFS folder targets, the existing departmental share was verified with PowerShell:

```
Get-SmbShare -Name CompanyShares | Select-Object Name, Path
```

The command confirmed:

Name	Path
------	------

----	----
------	------

CompanyShares	C:\BEN Industries_CompanyShares
---------------	---------------------------------

This established that the existing departmental data remained hosted on FILESRV01 and could be referenced by DFS.

Step 4 — Create DFS Namespace Folders

Created eight logical folders within the CorporateData namespace:

Executive

Finance

Human Resources

Information Technology

Marketing

Projects

Public

Sales

Each DFS folder was associated with its corresponding folder under the existing CompanyShares SMB share.

For example:

DFS Path:

\\benlab.local\CorporateData\Finance

Folder Target:

\\FILESRV01\CompanyShares\Finance

The same design was applied to the remaining departmental folders.

Step 5 — Verify the Namespace in DFS Management

DFS Management displayed all eight folders beneath:

`\\benlab.local\CorporateData`

This confirmed that the logical corporate namespace had been successfully constructed on FILESRV01.

Step 6 — Client-Side Validation from ADMINPC01

From the domain-joined ADMINPC01 workstation, File Explorer was used to access:

`\\benlab.local\CorporateData`

The workstation successfully displayed all eight DFS folders:

Executive

Finance

Human Resources

Information Technology

Marketing

Projects

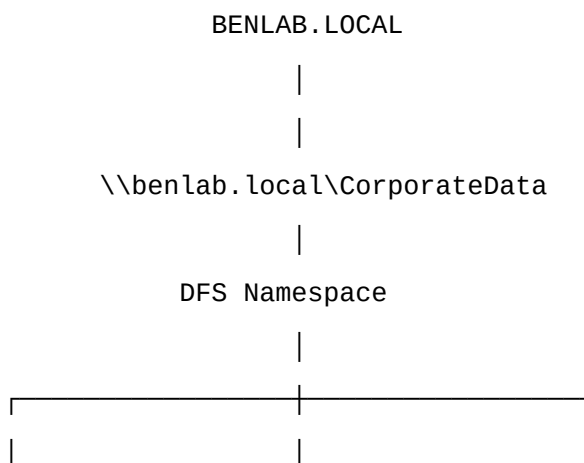
Public

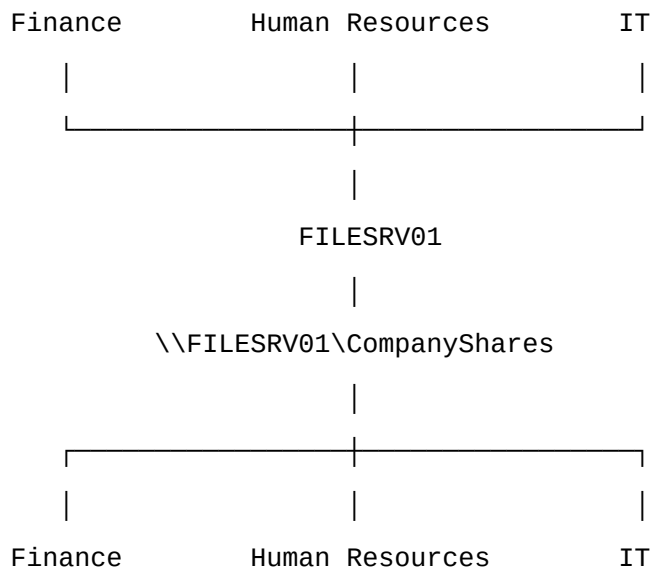
Sales

The Finance namespace had also previously been opened successfully and displayed the existing files stored in the underlying Finance folder.

This confirmed that DFS was correctly directing client access from the domain-based namespace to the physical folder target on FILESRV01.

Architecture





The same structure applies to Executive, Marketing, Projects, Public, and Sales.

Result

DFS Namespace configuration and client-side validation were successful.

The BENLAB environment now provides a centralized domain-based path:

`\\benlab.local\\CorporateData`

Users no longer need to know that the departmental data currently resides on FILESRV01.

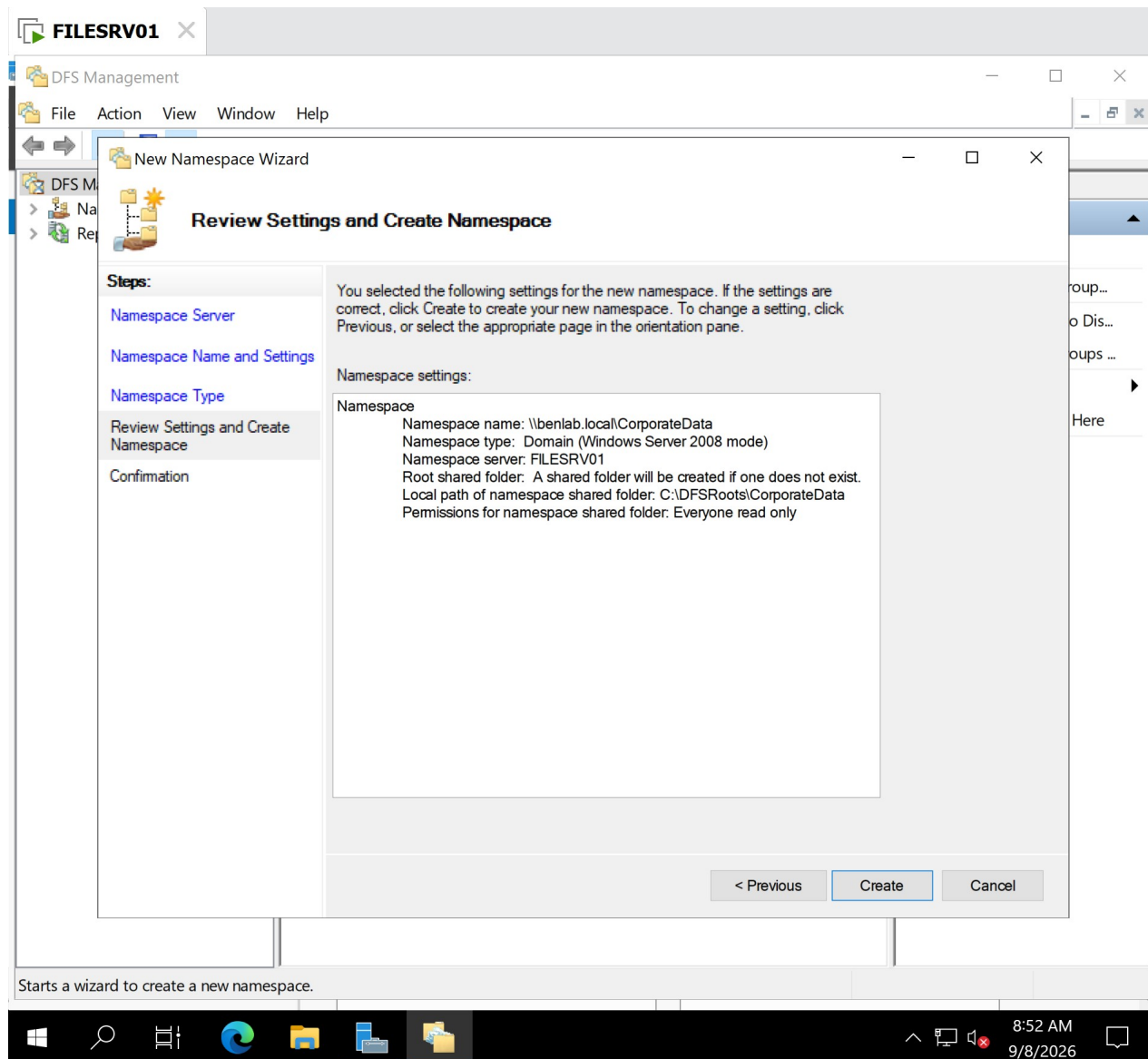
This introduces an important abstraction layer between the **user-facing file path** and the **physical server hosting the data**.

Key Learning Points

- **DFS Namespace (DFS-N)** provides a logical, centralized structure for accessing network shares.
- A **domain-based namespace** uses the Active Directory domain name rather than exposing a particular server name to users.
- A **DFS folder** is the logical folder users see, while a **folder target** identifies the actual SMB location containing the data.
- Existing SMB shares can be incorporated into DFS without moving the underlying files.
- DFS Namespace does **not** automatically copy or synchronize data.
- **DFS Replication (DFS-R)** is a separate technology used when data needs to be synchronized between multiple servers.

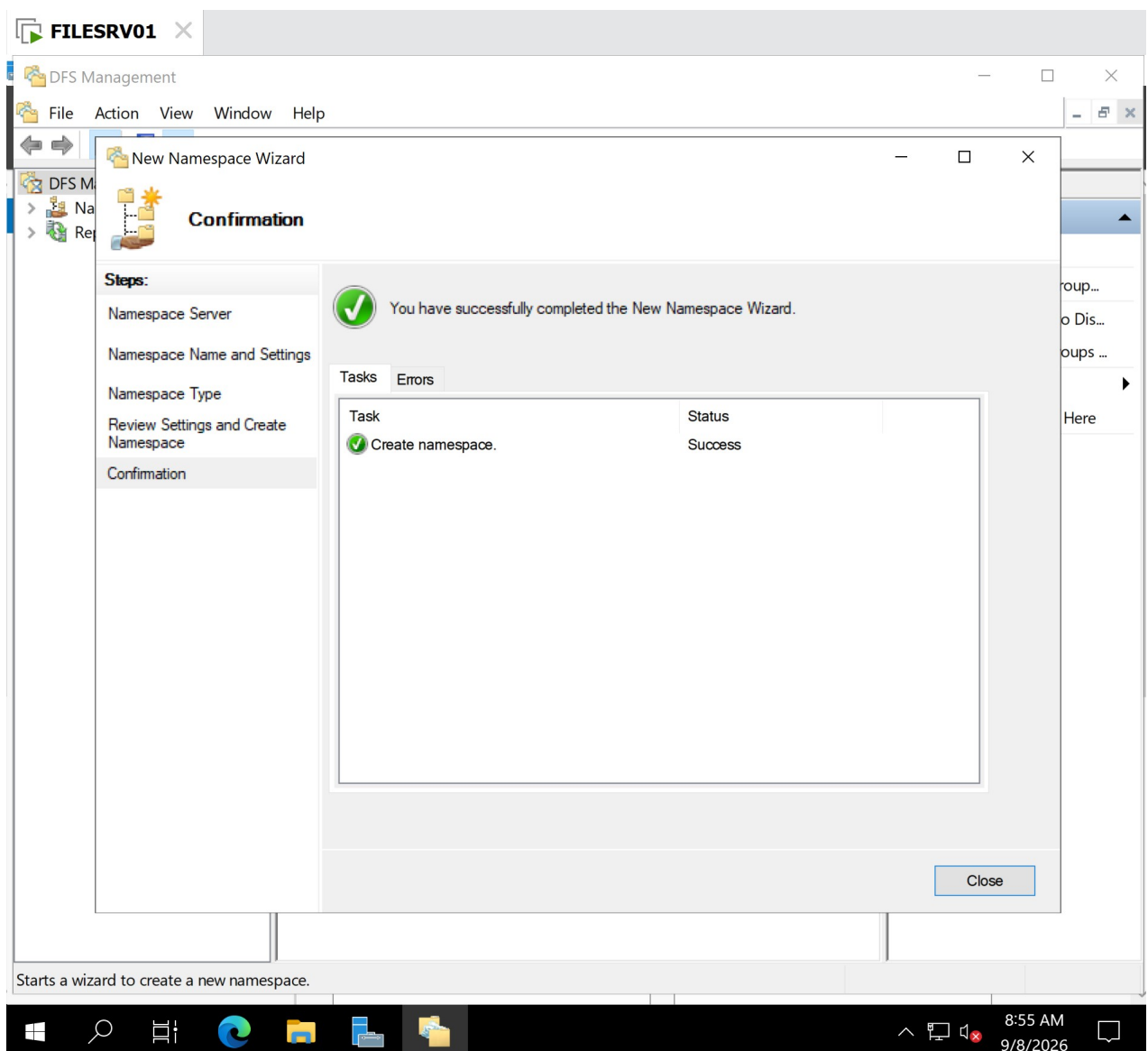
- Separating the logical namespace from the physical server makes future file-server migrations and multi-server designs easier because the user-facing path can remain consistent.

Screenshots / Evidence

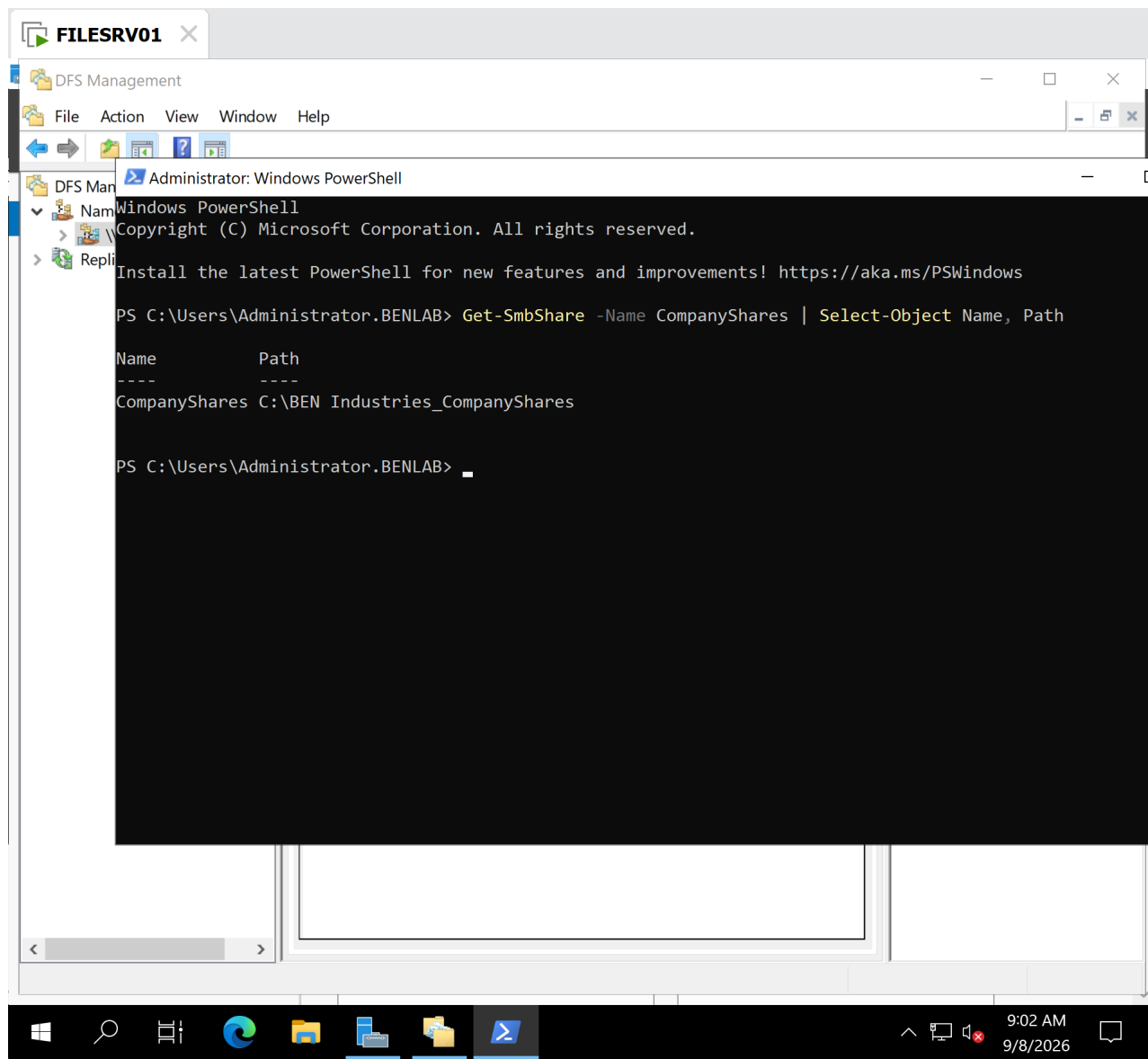


New Namespace Wizard — Namespace Type

Shows \\benlab.local\CorporateData configured as domain-based.

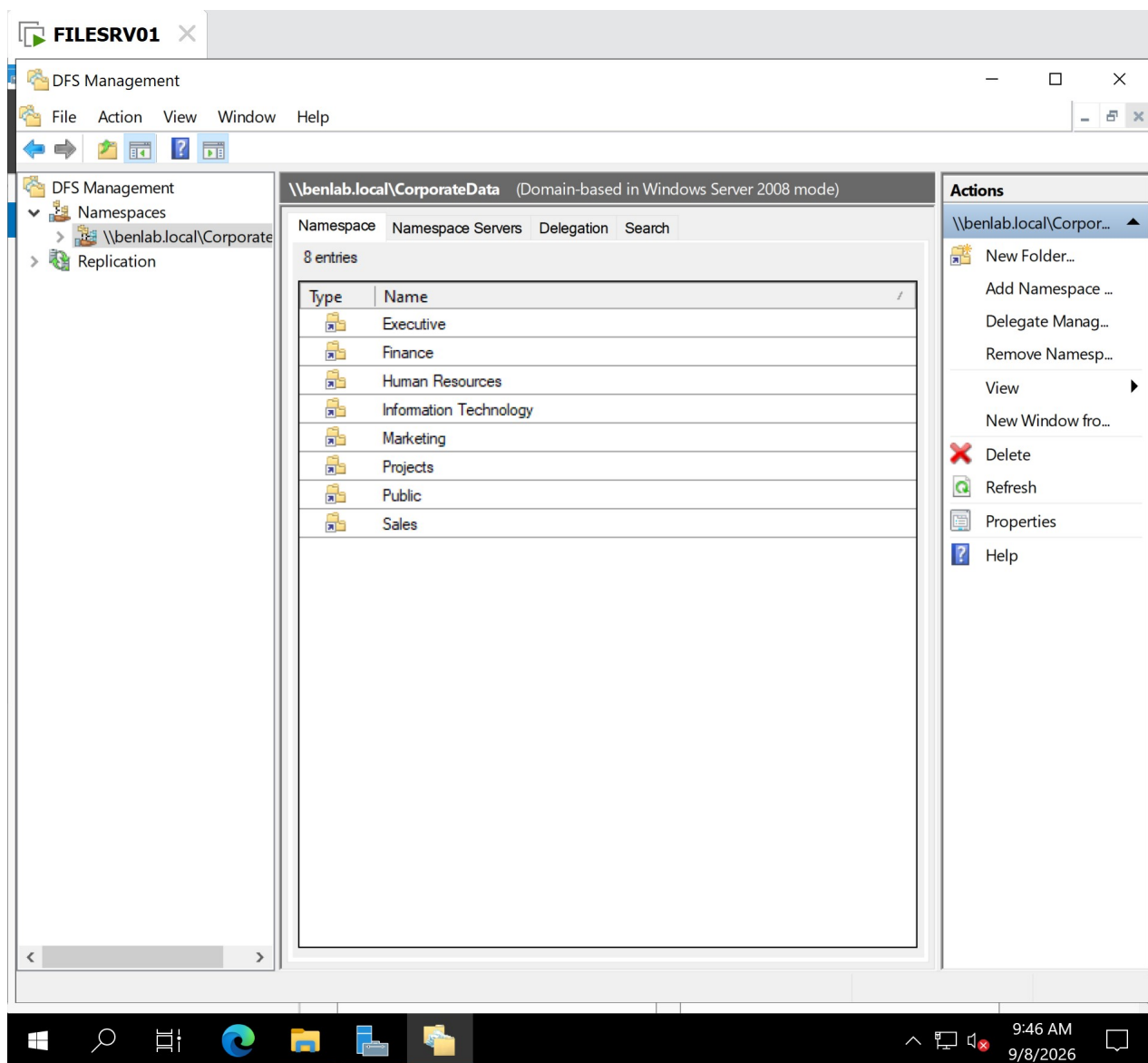


New Namespace Wizard — Successful Completion
Proves successful namespace creation.



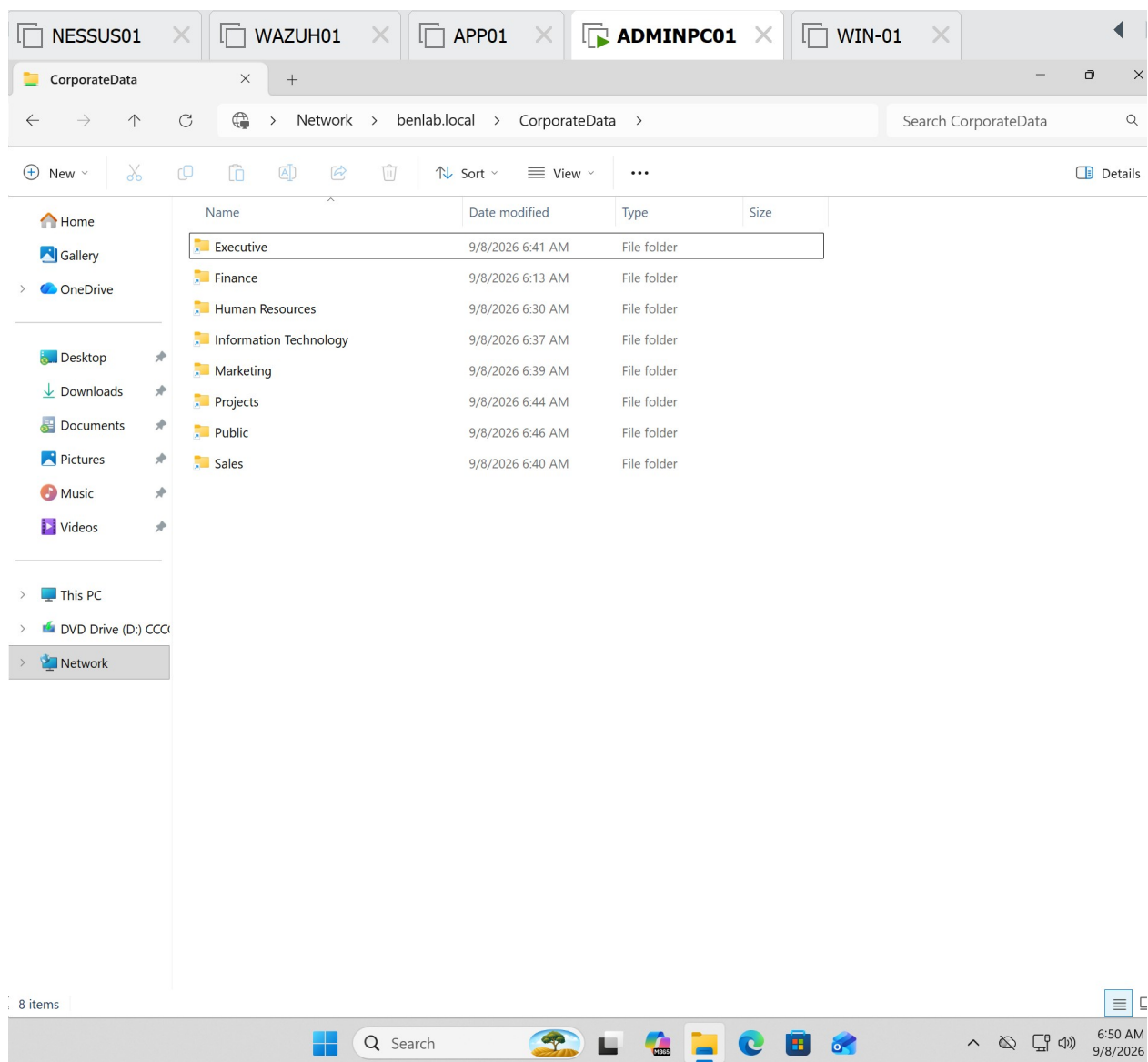
PowerShell — Existing CompanyShares Validation

Shows `Get -SmbShare` and the underlying `C:\BEN Industries_CompanyShares` path.



DFS Management — Eight Namespace Folders

Shows Executive through Sales under [\\benlab.local\CorporateData](#).



ADMINPC01 — Client-Side Namespace Validation ★

Showing all eight folders through Network > benlab.local > CorporateData.

Skills Demonstrated

Windows Server File Services • DFS Namespaces • Domain-Based DFS • SMB File Sharing • UNC Paths • Active Directory Integration • PowerShell Validation • Centralized File Access • Client-Side Testing

Task 8 – Configure and Validate DFS Replication

Objective

Configure **Distributed File System Replication (DFSR)** between the HQ file server **FILESRV01** and Branch file server **FILESRV02** to provide synchronized enterprise data between the two locations.

The task also demonstrates troubleshooting DFSR communication across routed networks protected by pfSense firewalls.

Lab Environment

HQ Site

- FILESRV01 — 10.20.20.50
- HQ network — 10.20.20.0/24
- Replicated folder — C:\EnterpriseData

Branch Site

- FILESRV02 — 10.30.30.101
- Branch network — 10.30.30.0/24
- Replicated folder — C:\EnterpriseData

Active Directory Domain

benlab.local

Replication Group

HQ-Branch-File-Replication

Replicated Folder

EnterpriseData

Step 1 – Validate DFS Replication Prerequisites

Verified that the DFS Namespace and DFS Replication role services were installed on both file servers.

PowerShell was used to validate the required Windows Server features.

The DFS Replication service was also confirmed to be running.

Step 2 – Configure the DFS Replication Group

A replication group named:

HQ-Branch-File-Replication

was configured with two members:

FILESRV01

FILESRV02

The replicated folder was configured as:

EnterpriseData

with the local path:

C:\EnterpriseData

on both servers.

Step 3 – Validate the Replication Configuration

PowerShell was used to verify the replication group:

```
Get-DfsReplicationGroup
```

The replication group returned:

GroupName : HQ-Branch-File-Replication

DomainName : benlab.local

State : Normal

Membership was then validated:

```
Get-DfsrMember -GroupName "HQ-Branch-File-Replication"
```

Both servers appeared as members:

FILESRV01

FILESRV02

with their DFSR state reported as:

Normal

The replicated folder configuration was also validated. Both servers showed:

FolderName : EnterpriseData

ContentPath : C:\EnterpriseData

Enabled : True

State : Normal

Step 4 – Troubleshoot DFSR Network Connectivity

Because FILESRV01 and FILESRV02 reside on different routed networks, firewall and network connectivity had to be validated.

Connectivity was tested with:

```
Test-NetConnection 10.30.30.101 -Port 5722
```

and in the opposite direction:

```
Test-NetConnection 10.20.20.50 -Port 5722
```

The final result was:

```
TcpTestSucceeded : True
```

This confirmed TCP communication for the configured DFSR service between the HQ and Branch file servers.

pfSense firewall rules were configured to permit the required inter-site communication.

Step 5 – Force DFSR Configuration Refresh

The servers were instructed to immediately retrieve updated DFSR configuration information from Active Directory:

```
dfsrdiag pollad
```

Result:

```
Operation Succeeded
```

Replication activity was also inspected using:

```
dfsrdiag replicationstate
```

This provided visibility into active DFSR inbound and outbound replication operations.

Step 6 – Validate HQ-to-Branch Replication

A test file was created on **FILESRV01** inside:

```
C:\EnterpriseData
```

The contents of the replicated directory were checked using:

```
Get-ChildItem C:\EnterpriseData
```

The test file subsequently appeared on **FILESRV02**, demonstrating successful:

FILESRV01

↓

FILESRV02

HQ-to-Branch replication.

Step 7 – Validate Branch-to-HQ Replication

A second test file was created directly on FILESRV02:

```
"Created on FILESRV02 to validate Branch-to-HQ DFS Replication" |
```

```
Out-File C:\EnterpriseData\DFSR-Test-Branch.txt
```

The directory was checked:

```
Get-ChildItem C:\EnterpriseData
```

DFSR-Test-Branch.txt subsequently appeared on FILESRV01.

This demonstrated successful replication in the opposite direction:

FILESRV02

↓

FILESRV01

Therefore, **bidirectional DFS Replication was successfully demonstrated.**

Step 8 – Validate DFSR Backlog

The DFSR backlog was checked with:

```
dfsrdiag backlog /rname:"HQ-Branch-File-Replication" /rfname:"EnterpriseData"  
/smem:FILESRV01 /rmem:FILESRV02
```

A successful test returned:

```
No Backlog - member <filesrv02> is in sync with partner <filesrv01>
```

Operation Succeeded

This demonstrated that the two DFSR members had reached synchronization during validation.

Troubleshooting – WMI/RPC Remote Management

Additional `dfsrdiag backlog` tests exposed a separate remote-management issue.

The command sometimes returned:

```
[ERROR] Failed to connect to WMI services on computer
```

Operation Failed

Testing showed that basic network connectivity remained available.

RPC Endpoint Mapper connectivity was tested using:

```
Test-NetConnection FILESRV02 -Port 135
```

and eventually returned:

```
TcpTestSucceeded : True
```

Windows Management Instrumentation firewall rules were examined using:

```
Get-NetFirewallRule -DisplayGroup "Windows Management Instrumentation (WMI)" |  
Select-Object DisplayName, Enabled, Direction, Action
```

The WMI firewall rules were enabled where required:

```
Enable-NetFirewallRule -DisplayGroup "Windows Management Instrumentation (WMI)"
```

Further investigation showed that Windows was listening on dynamic RPC ports in the range:
49152–65535

using:

```
Get-NetTCPConnection -State Listen |  
Where-Object {$_.LocalPort -ge 49152 -and $_.LocalPort -le 65535} |  
Select-Object LocalAddress, LocalPort, OwningProcess
```

A direct IPv4 test to one of these ports demonstrated that additional RPC/WMI communication across the routed environment remained restricted:

```
Test-NetConnection 10.30.30.101 -Port 49664
```

Result:

```
PingSucceeded      : True  
TcpTestSucceeded  : False
```

This issue affected some **remote WMI/RPC-based diagnostic operations** but did not invalidate the successfully demonstrated DFSR file replication.

Important Ports Encountered

Protocol/Service	Port	Purpose
SMB	TCP 445	Windows file sharing and administrative communication
RPC Endpoint Mapper	TCP 135	Initial RPC service discovery
Dynamic RPC	TCP 49152–65535	Dynamic Windows RPC communication
DFS Replication	TCP 5722	DFSR communication used in this lab

These ports should also be added to the **Windows Administrator Field Reference**.

Validation Results

Replication Group: HQ-Branch-File-Replication — Normal

FILESRV01 membership: Normal

FILESRV02 membership: Normal

EnterpriseData replicated folder: Enabled / Normal

HQ → Branch replication: Successful

Branch → HQ replication: Successful

DFSR backlog synchronization: Successfully demonstrated

TCP 5722 connectivity: Successful

Remote WMI/RPC diagnostics: Additional troubleshooting required

Key Learning Points

This task demonstrated that DFS Replication is more than simply creating a replication group. Successful deployment requires coordination between **Active Directory, DFSR configuration, Windows services, routing, Windows Firewall, and network firewalls.**

A particularly important troubleshooting lesson was the distinction between the **data replication path** and the **management/diagnostic path**. DFSR successfully synchronized files between FILESRV01 and FILESRV02 even though some later WMI-based diagnostic commands encountered RPC connectivity problems.

The troubleshooting process also reinforced a useful administrative methodology:

Verify IP connectivity

↓

Verify required service port

↓

Verify Windows service is listening

↓

Verify Windows Firewall

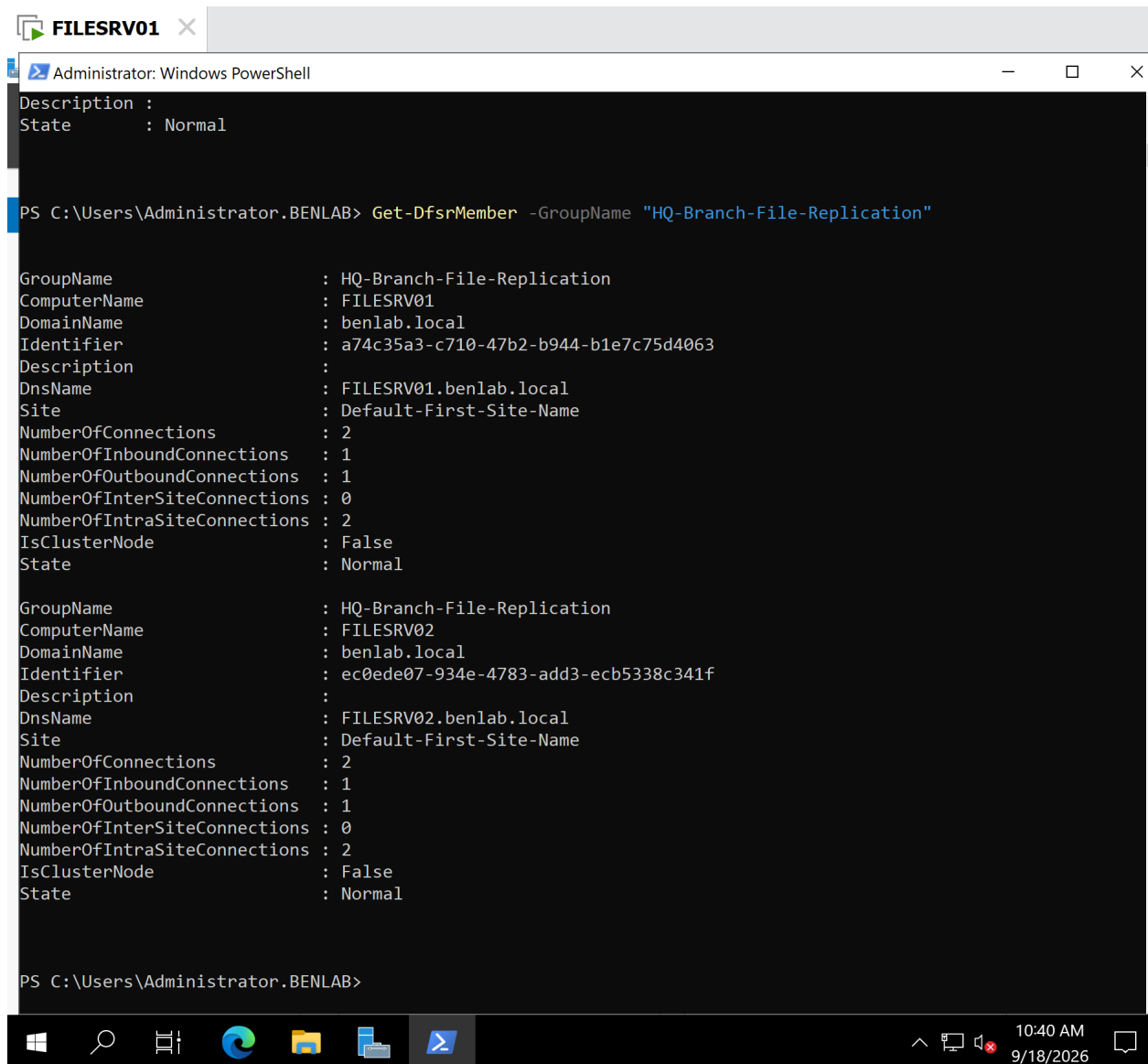
↓

Verify pfSense/firewall path

↓

Verify application/service operation

Screenshots / Evidence



```
FILESRV01 x
Administrator: Windows PowerShell
Description :
State      : Normal

PS C:\Users\Administrator.BENLAB> Get-DfsrMember -GroupName "HQ-Branch-File-Replication"

GroupName      : HQ-Branch-File-Replication
ComputerName   : FILESRV01
DomainName     : benlab.local
Identifier     : a74c35a3-c710-47b2-b944-b1e7c75d4063
Description    :
DnsName        : FILESRV01.benlab.local
Site           : Default-First-Site-Name
NumberOfConnections : 2
NumberOfInboundConnections : 1
NumberOfOutboundConnections : 1
NumberOfInterSiteConnections : 0
NumberOfIntraSiteConnections : 2
IsClusterNode  : False
State          : Normal

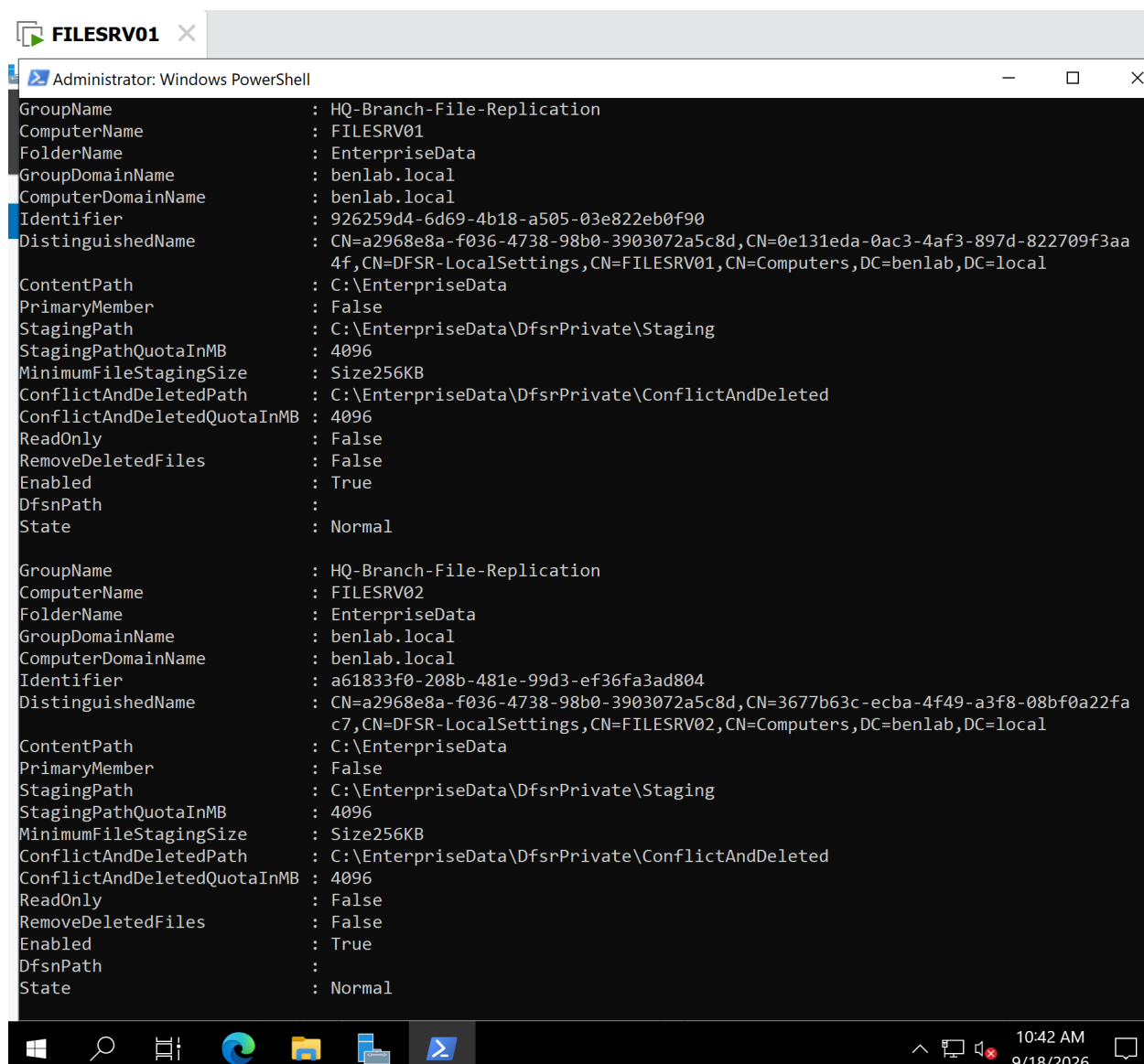
GroupName      : HQ-Branch-File-Replication
ComputerName   : FILESRV02
DomainName     : benlab.local
Identifier     : ec0ede07-934e-4783-add3-ecb5338c341f
Description    :
DnsName        : FILESRV02.benlab.local
Site           : Default-First-Site-Name
NumberOfConnections : 2
NumberOfInboundConnections : 1
NumberOfOutboundConnections : 1
NumberOfInterSiteConnections : 0
NumberOfIntraSiteConnections : 2
IsClusterNode  : False
State          : Normal

PS C:\Users\Administrator.BENLAB>
```

Screenshot 1 – DFS Replication Members Validated

Caption:

PowerShell validation showing FILESRV01 and FILESRV02 successfully configured as members of the HQ-Branch-File-Replication replication group with a Normal state.

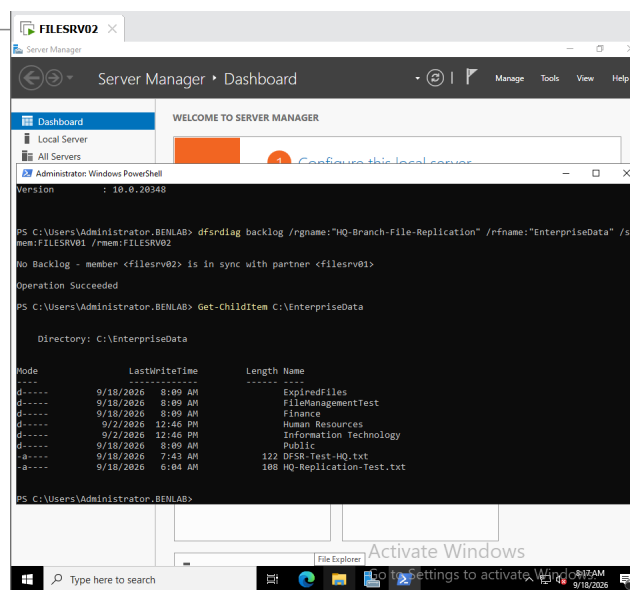


Screenshot 2 – DFS Replicated Folder Configuration

Caption:

DFSR configuration validation confirming that C:\EnterpriseData is enabled as the replicated folder on both FILESRV01 and FILESRV02.

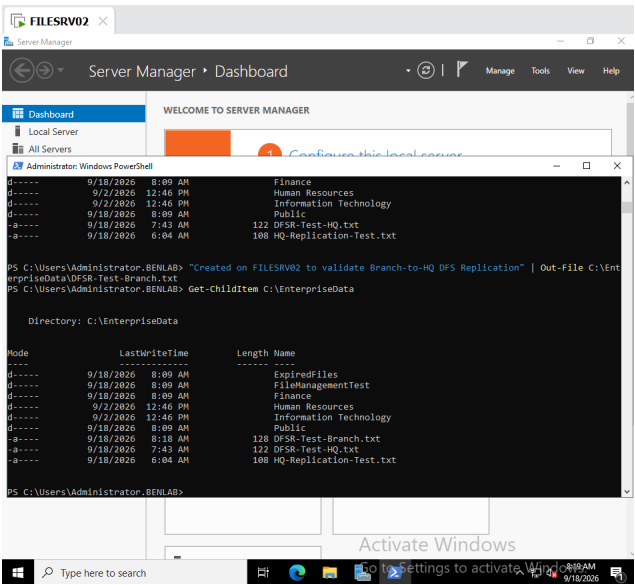
Screenshot 3 – DFSR Backlog Successfully Synchronized

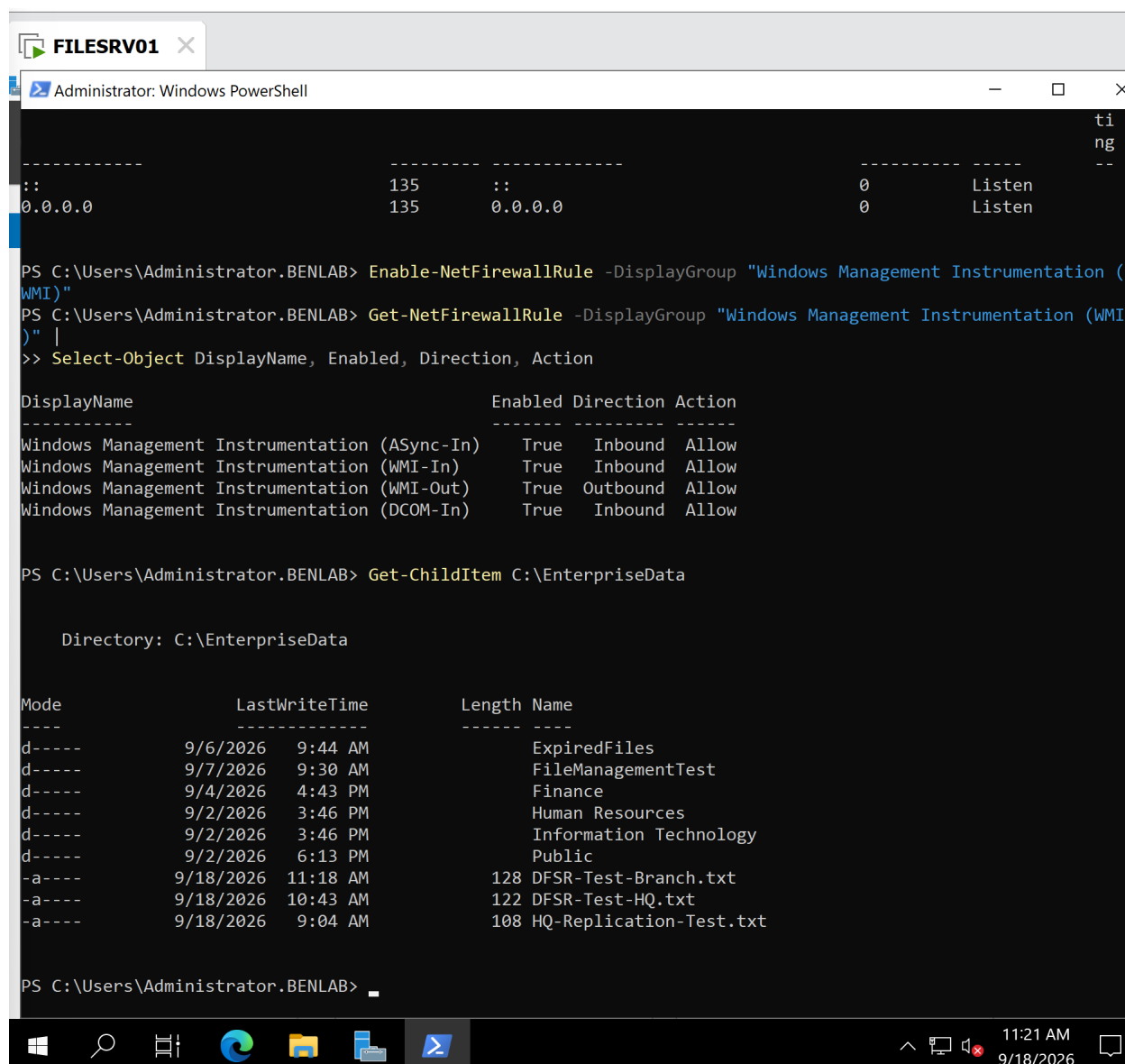


Caption:
DFSR backlog validation confirming that FILESRV02 was synchronized with FILESRV01 with no pending replication backlog.

**Screenshot 4 –
Branch-to-HQ Test
File Created on
FILESRV02**

Caption:
Test file created on
FILESRV02 to validate
Branch-to-HQ DFS
Replication.



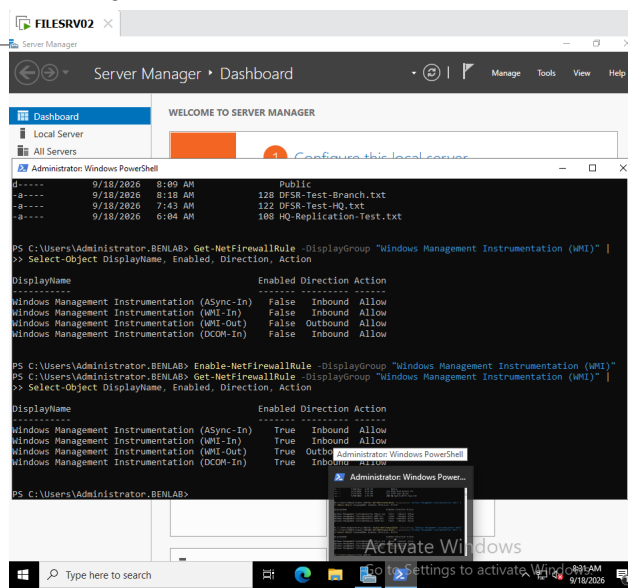


Screenshot 5 – Branch File Successfully Replicated to FILESRV01

Caption:

FILESRV01 showing DFSR-Test-Branch.txt, confirming successful replication of a file created at the Branch server back to the HQ server.

Screenshot 6 – WMI Firewall Rules Enabled

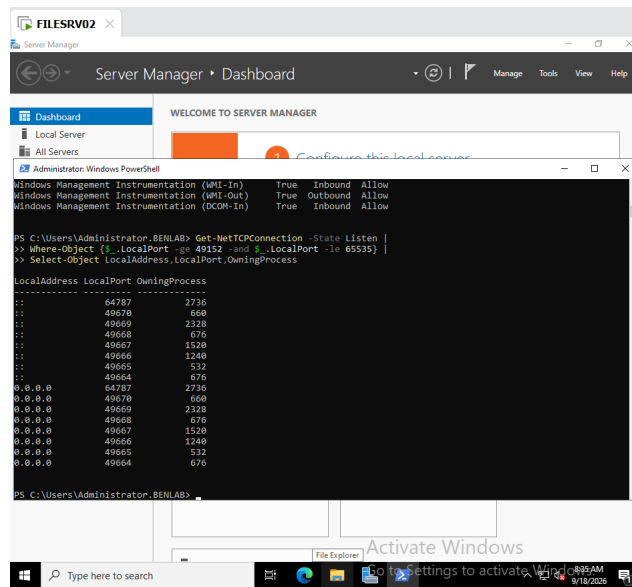


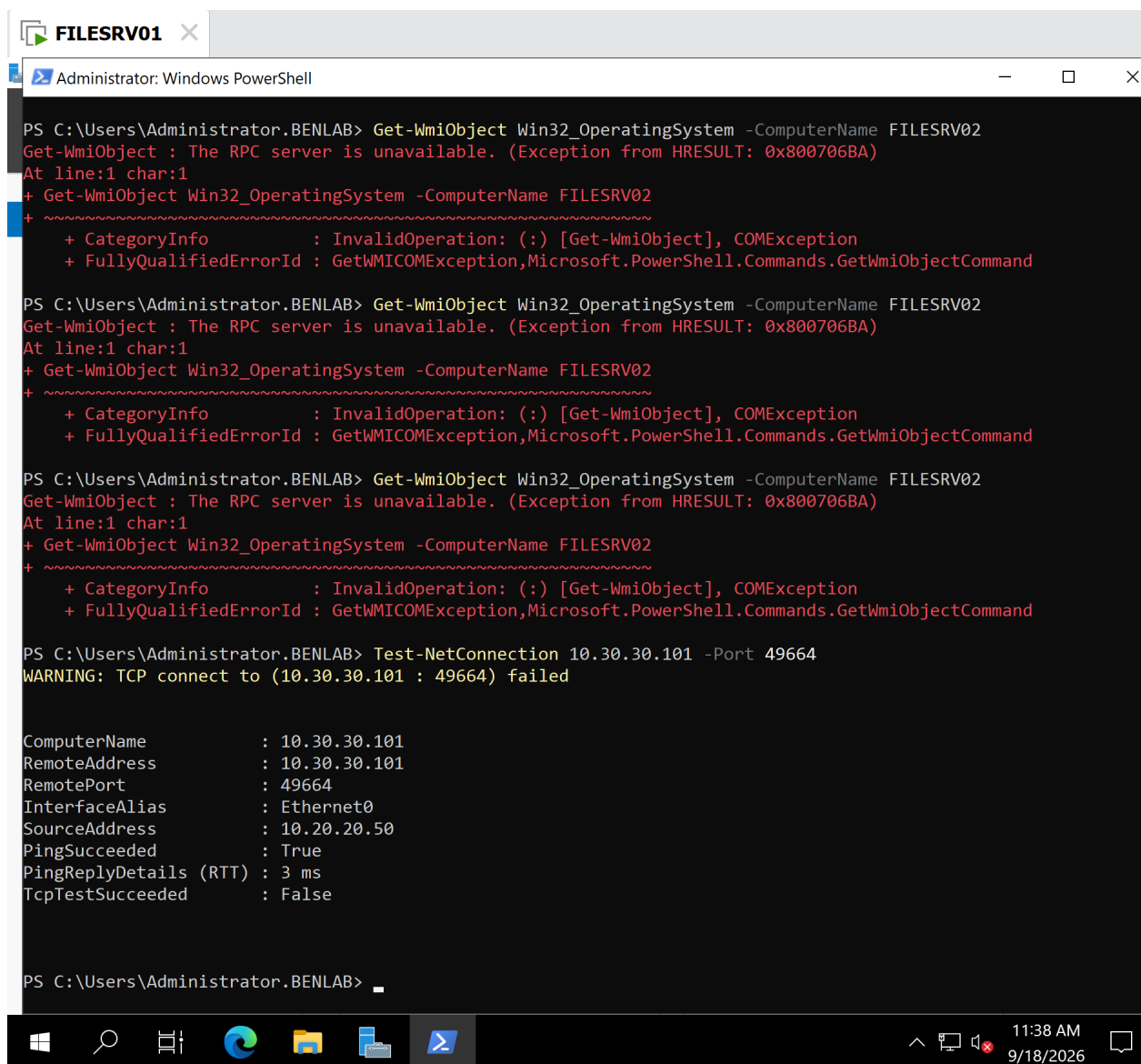
Caption:

Windows Firewall WMI rules enabled on FILESRV02 while troubleshooting remote DFSR backlog management.

**Screenshot 7 –
Dynamic RPC Ports
Identified****Caption:**

Dynamic RPC listening ports identified on FILESRV02 while investigating WMI/RPC communication across the routed HQ-to-Branch network.





```
PS C:\Users\Administrator.BENLAB> Get-WmiObject Win32_OperatingSystem -ComputerName FILESRV02
Get-WmiObject : The RPC server is unavailable. (Exception from HRESULT: 0x800706BA)
At line:1 char:1
+ Get-WmiObject Win32_OperatingSystem -ComputerName FILESRV02
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [Get-WmiObject], COMException
+ FullyQualifiedErrorId : GetWmiCOMException,Microsoft.PowerShell.Commands.GetWmiObjectCommand

PS C:\Users\Administrator.BENLAB> Get-WmiObject Win32_OperatingSystem -ComputerName FILESRV02
Get-WmiObject : The RPC server is unavailable. (Exception from HRESULT: 0x800706BA)
At line:1 char:1
+ Get-WmiObject Win32_OperatingSystem -ComputerName FILESRV02
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [Get-WmiObject], COMException
+ FullyQualifiedErrorId : GetWmiCOMException,Microsoft.PowerShell.Commands.GetWmiObjectCommand

PS C:\Users\Administrator.BENLAB> Get-WmiObject Win32_OperatingSystem -ComputerName FILESRV02
Get-WmiObject : The RPC server is unavailable. (Exception from HRESULT: 0x800706BA)
At line:1 char:1
+ Get-WmiObject Win32_OperatingSystem -ComputerName FILESRV02
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (:) [Get-WmiObject], COMException
+ FullyQualifiedErrorId : GetWmiCOMException,Microsoft.PowerShell.Commands.GetWmiObjectCommand

PS C:\Users\Administrator.BENLAB> Test-NetConnection 10.30.30.101 -Port 49664
WARNING: TCP connect to (10.30.30.101 : 49664) failed

ComputerName      : 10.30.30.101
RemoteAddress     : 10.30.30.101
RemotePort        : 49664
InterfaceAlias    : Ethernet0
SourceAddress     : 10.20.20.50
PingSucceeded     : True
PingReplyDetails (RTT) : 3 ms
TcpTestSucceeded  : False

PS C:\Users\Administrator.BENLAB>
```

Screenshot 8 – RPC Dynamic Port Blocked Across Sites

Caption:

Network troubleshooting demonstrating successful IP connectivity between HQ and Branch while TCP 49664 remained inaccessible, isolating the remaining issue to RPC/firewall communication rather than basic routing.

Final Result

Task 8 – Configure and Validate DFS Replication: COMPLETED

A bidirectional DFS Replication environment was successfully configured between **HQ FILESRV01** and **Branch FILESRV02**. Files created at either site were successfully replicated to the opposite server.

An additional WMI/RPC management limitation across the routed environment was identified and documented as part of the troubleshooting process rather than being treated as a failure of DFS Replication itself.
